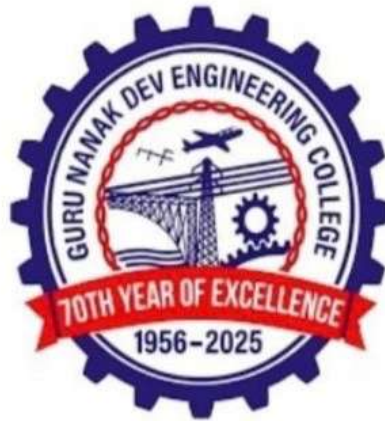


FINAL REPORT
Reconnaissance and venerability tester using AI

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF THE
DEGREE OF

BACHELOR OF TECHNOLOGY
(Computer Science and Engineering)



Submitted By:

Name: Jaskaran Singh Minhas

URN: 2302722

Name: Chirag Ahluwalia

URN: 2203420

Mentor:

Pf.Shailja

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GURU NANAK DEV ENGINEERIG COLLEGE LUDHIANA
(An Autonomous College Under UGC ACT)
Ludhiana 141006

Index

Sr No.	Contents	Page No.
1	Introduction	2
2	Obejctives	3
3	System Requirement - (Hardware/Software)	4-6
4	Software Requirement Analysis	7-8
5	Software Design	9-15
6	AI-Driven Security Testing and Vulnerability Analysis	16-24
7	Testing Module	25-28
8	Performance of the project Developed	29-33
9	Output Screens	34-36
10	References	37

INTRODUCTION

With the rapid growth of internet technologies and online services, cybersecurity has become a critical concern for organizations and individuals. Websites and web applications are constantly exposed to cyber threats such as data breaches, unauthorized access, malware attacks, and exploitation of vulnerabilities. Therefore, identifying weaknesses in systems before attackers exploit them is extremely important.

One of the first phases of cybersecurity testing is **reconnaissance**, where information about the target system is gathered. This process includes discovering domains and subdomains, scanning open ports, identifying technologies used by websites, and searching for hidden directories that may expose sensitive data.

Traditionally, cybersecurity professionals use multiple tools separately from the command line to perform these tasks. This process can be time-consuming and complex, especially for beginners.

The project “**Reconnaissance and Vulnerability Tester Using AI**” aims to automate the reconnaissance and vulnerability testing process using artificial intelligence and automated security tools. The system integrates several security tools such as **Subfinder, Nmap, WhatWeb, and Dirsearch** within a single framework.

The framework is developed using **Python** and provides a **web-based interface using Streamlit**, allowing users to enter a target domain and automatically perform security scans. The system collects results from multiple modules and uses AI-assisted analysis to identify potential vulnerabilities and determine whether the target domain is secure.

The rapid expansion of the global digital economy has forced organizations to migrate their core infrastructures, customer portals, and internal workflows directly onto web applications and cloud architectures.

While this digital evolution has accelerated business efficiency, it has simultaneously introduced an unprecedented era of systemic risk. The modern internet landscape is a hostile environment where public-facing web applications are subjected to unceasing, automated scanning campaigns conducted by malicious actors.

These adversaries actively hunt for low-hanging fruit, zero-day vulnerabilities, and architectural misconfigurations to orchestrate devastating cyber attacks. Data breaches that compromise millions of records, unauthorized access to sensitive financial environments, stealthy ransomware deployments, and targeted malware injections are no longer isolated incidents—they represent systemic threats to organizational survival. Because modern enterprise networks are highly distributed, securing them requires a fundamental shift away from legacy, reactive defense protocols. Security teams cannot afford to wait for a high-priority incident alert or an active compromise to initiate a defense response. Instead, sustainable risk management requires proactive, continuous security testing designed to identify, analyze, and remediate technical vulnerabilities before an

external threat actor can detect and exploit them. The logical baseline of any proactive defensive posture is cybersecurity reconnaissance, which represents the initial, foundational phase of an attack surface audit.

Before an engineering team can defend a system—or before an ethical hacker can evaluate its resilience—they must fully map the target's external digital footprint. This data-gathering phase simulates the exact footprinting steps an adversary takes during the initial stages of an attack campaign.

Effective reconnaissance requires exploring multiple computational layers to expose hidden vulnerabilities. It begins with subdomain discovery, which systematically harvests peripheral web addresses linked to a primary domain to locate forgotten staging environments, legacy portals, or employee log-ins that might bypass primary firewall rules.

Simultaneously, active port scanning probes the host network to identify open communication channels, mapping out running services and listening protocols.

This is followed immediately by technology fingerprinting, a process that reads HTTP header responses, cookies, and source code parameters to determine the exact software versions, backend frameworks, and content management systems (CMS) powering the target.

Finally, automated directory traversal tools brute-force the web infrastructure using extensive wordlists to uncover unlinked files, hidden development directories, and exposed backups that may inadvertently leak administrative credentials or source configurations to the public.

Cybersecurity professionals have historically relied on a disparate toolkit of command-line interface (CLI) utilities to evaluate a target's architecture. An analyst must manually launch independent tools inside separate terminal windows—using utilities like Subfinder to gather subdomains, Nmap to probe network ports, WhatWeb to fingerprint server software, and Dirsearch to hunt for hidden directories.

Each utility operates on its own runtime logic, demands specialized command-line switches, and generates raw, unstructured text files or terminal streams in varying outputs. This legacy workflow introduces major operational friction: it is incredibly time-consuming, highly prone to human oversight, and scales poorly across massive corporate networks.

For junior administrators or security beginners, the steep learning curve required to master terminal configurations and manually correlate thousands of lines of chaotic log files often results in critical, exploitable vulnerabilities slipping through the cracks unnoticed.

To resolve these operational inefficiencies, the unified framework “Reconnaissance and Vulnerability Tester Using AI” automates the entire discovery pipeline.

Developed in Python and delivered through a streamlined, web-based Streamlit dashboard, this application abstracts away terminal complexities into a single-click security assessment interface.

When a user inputs a target URL or domain into the control panel, the Python-driven backend orchestrates the simultaneous, asynchronous execution of Subfinder, Nmap, WhatWeb, and Dirsearch in the background. The platform dynamically manages the execution lifecycles of these utilities, prevents system resource locks, and funnels their raw terminal text captures directly into an intelligent, centralized data processing layer.

By unifying these powerful utilities into a singular, cohesive pipeline, the system eliminates the administrative overhead of tool management, vastly accelerates scanning speeds, and ensures that all collected perimeter data is securely aggregated without any risk of human correlation error. The true core innovation of this platform lies within its AI-Assisted Analysis Layer, which transforms raw network data into contextual, actionable defense intelligence.

Traditional automated scanning systems are fundamentally limited by signature-based rules; they compare software version strings against rigid, pre-defined databases and completely fail to recognize complex, novel threats.

Our framework overcomes this limitation by transforming the parsed, structured JSON payload of the entire attack surface and passing it into a highly specialized Artificial Intelligence engine. Acting as an on-demand virtual penetration testing expert, the AI model applies deep context-driven reasoning to evaluate how minor, isolated findings across different tools can be chained together by an attacker to create a critical exploit path.

Furthermore, the generative AI engine eliminates the industry problem of alert fatigue by distinguishing genuine, exploitable vulnerabilities from benign configuration properties.

It automatically calculates localized risk scores and outputs clear, step-by-step remediation playbooks—complete with necessary patch recommendations, configuration modifications, and protective firewall rules—allowing system administrators of all skill levels to defend their networks with enterprise-grade precision.

The project focuses on achieving the following:

Conducting reconnaissance on target domains to identify possible vulnerabilities.

Scanning open ports and services to detect potential security risks.

Detecting technologies used by web applications and identifying outdated or vulnerable components.

Discovering hidden directories and files that may expose sensitive information.

Why is cyber security important?

Listed below are the reasons why cyber security is so important in what's become a predominant digital world

- Cyber attacks can be extremely expensive for businesses to endure.
- In addition to financial damage suffered by the business, a data breach can also inflict untold reputational damage.
- Cyber-attacks these days are becoming progressively destructive. Cybercriminals are using more sophisticated ways to initiate cyber attacks.

Cyber security Fundamentals –

Confidentiality: Confidentiality is about preventing the disclosure of data to unauthorized parties. It also means trying to keep the identity of authorized parties involved in sharing and holding data private and anonymous. Often confidentiality is compromised by cracking poorly encrypted data, Man-in-the-middle (MITM) attacks, disclosing sensitive data. Standard measures to establish confidentiality include:

- Data encryption
 - Two-factor authentication
 - Biometric verification
 - Security tokens
- Integrity** Integrity refers to protecting information from being modified by unauthorized parties. Standard measures to guarantee integrity include:

- Cryptographic checksums
- Using file permissions
- Uninterrupted power supplies
- Data backups

OBJECTIVES

Through the use of automated security tools and AI-assisted analysis, the project aims to create an efficient system for performing reconnaissance and vulnerability testing on target domains.

Specific objectives include:

Creating an AI-based setup for identifying and analyzing target domains.

Using AI-assisted techniques to scan and identify open ports of discovered domains.

Performing AI-driven security testing to detect vulnerabilities and analyze results to determine whether the domain is secure or not.

Types of Cyber Attacks

A cyber-attack is an exploitation of computer systems and networks. It uses malicious code to alter computer code, logic or data and lead to cybercrimes, such as information and identity theft.

Cyber-attacks can be classified into the following categories:

- 1) Web-based attacks
- 2) System-based attacks

Web-based attacks

These are the attacks which occur on a website or web applications.

Some of the important web-based attacks are as follows-

1. Injection attacks

It is the attack in which some data will be injected into a web application to manipulate the application and fetch the required information.

Example- SQL Injection, code Injection, log Injection, XML Injection etc.

2. DNS Spoofing

DNS Spoofing is a type of computer security hacking. Whereby a data is introduced into a DNS resolver's cache causing the name server to return an incorrect IP address, diverting traffic to the attackers computer or any other computer.

The DNS spoofing attacks can go on for a long period of time without being detected and can cause serious security issues.

3. Session Hijacking

It is a security attack on a user session over a protected network. Web applications create cookies to store the state and user sessions.

By stealing the cookies, an attacker can have access to all of the user data.

4. Phishing Phishing

is a type of attack which attempts to steal sensitive information like user login credentials and credit card number. It occurs when an attacker is masquerading as a trustworthy entity in electronic communication.

5. Brute force.

It is a type of attack which uses a trial and error method. This attack generates a large number of guesses and validates them to obtain actual data like user password and personal identification number.

This attack may be used by criminals to crack encrypted data, or by security, analysts to test an organization's network security.

6. Denial of Service

It is an attack which meant to make a server or network resource unavailable to the users. It accomplishes this by flooding the target with traffic or sending it information that triggers a crash. It uses the single system and single internet connection to attack a server.

It can be classified into the following-

Volume-based attacks-

Its goal is to saturate the bandwidth of the attacked site, and is measured in bit per second. Protocol attacks- It consumes actual server resources, and is measured in a packet.

Application layer attacks

- Its goal is to crash the web server and is measured in request per second.

7. Dictionary attacks

This type of attack stored the list of a commonly used password and validated them to get original password

8. URL Interpretation

It is a type of attack where we can change the certain parts of a URL, and one can make a web server to deliver web pages for which he is not authorized to browse.

9. File Inclusion attacks

It is a type of attack that allows an attacker to access unauthorized or essential files which is available on the web server or to execute malicious files on the web server by making use of the include functionality.

10. Man in the middle attacks

It is a type of attack that allows an attacker to intercepts the connection between client and server and acts as a bridge between them. Due to this, an attacker will be able to read, insert and modify the data in the intercepted connection.

System-based attacks

These are the attacks which are intended to compromise a computer or a computer network. Some of the important

system-based attacks are as follows-

1. Virus

It is a type of malicious software program that spread throughout the computer files without the knowledge of a user. It is a self-replicating malicious computer program that replicates by inserting copies of itself into other computer programs when executed. It can also execute instructions that cause harm to the system.

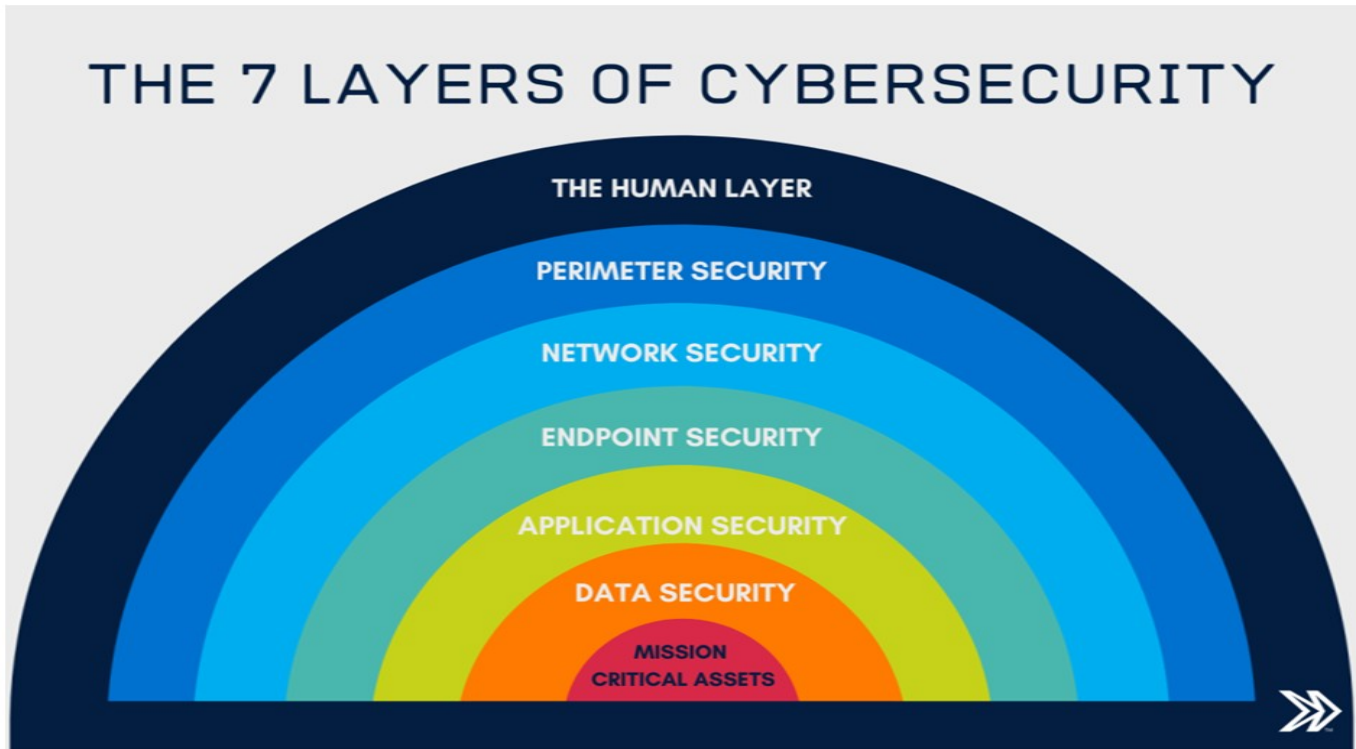
2. Worm

It is a type of malware whose primary function is to replicate itself to spread to uninfected computers. It works same as the computer virus. Worms often originate from email attachments that appear to be from trusted senders.

3. Trojan horse

It is a malicious program that occurs unexpected changes to computer setting and unusual activity, even when the computer should be idle. It misleads the user of its true intent. It appears to be a normal application but when opened/executed some malicious code will run in the background.

7 LAYERS OF CYBERSECURITY



- 1: Mission Critical Assets – This is the data you need to protect
- 2: Data Security – Data security controls protect the storage and transfer of data.
- 3: Application Security – Applications security controls protect access to an application, an application's access to your mission critical assets, and the internal security of the application.
- 4: Endpoint Security – Endpoint security controls protect the connection between devices and the network.
- 5: Network Security – Network security controls protect an organization's network and prevent unauthorized access of the network.
- 6: Perimeter Security – Perimeter security controls include both the physical and digital security methodologies that protect the business overall.
- 7: The Human Layer – Humans are the weakest link in any cyber security posture. Human security controls include phishing simulations and access management controls that protect mission critical assets from a wide variety of human threats, including cyber criminals, malicious insiders, and negligent users.

Cyber threats are security incidents or circumstances with the potential to have a negative outcome for your network or other data management systems.

Examples of common types of security threats include phishing attacks that result in the installation of malware that infects your data, failure of a staff member to follow data protection protocols that cause a data breach, or even a tornado that takes down your company's data headquarters, disrupting access.

Vulnerabilities are the gaps or weaknesses in a system that make threats possible and tempt threat actors to exploit them. Types of vulnerabilities in network security include but are not limited to SQL injections, server misconfigurations, cross-site scripting, and transmitting sensitive data in a non encrypted plain text format. When threat probability is multiplied by the potential loss that may result, cyber security experts, refer to this as a risk.

SECURITY VULNERABILITIES, THREATS AND ATTACKS

– Categories of vulnerabilities

- Corrupted (Loss of integrity)
- Leaky (Loss of confidentiality)
- Unavailable or very slow (Loss of availability) – Threats represent potential security harm to an asset when

vulnerabilities are exploited - Attacks are threats that have been carried out

- Passive – Make use of information from the system without affecting system resources
- Active – Alter system resources or affect operation
- Insider – Initiated by an entity inside the organization
- Outsider – Initiated from outside the perimeter

System Requirements

Hardware Requirements:

Guest Operating System:

- Windows 10 / Windows 11 with **VS Code (latest version)** for development.

Reconnaissance and Vulnerability Testing Tools:

- **Subfinder (latest version)** — used for discovering subdomains of the target domain during the reconnaissance phase.
- **Nmap (latest stable release)** — used for scanning open ports and identifying services running on the target system.
- **WhatWeb** — used for detecting technologies used by the target website such as web servers, CMS, and programming frameworks.
- **Dirsearch** — used for discovering hidden directories and files on the target website.

Supporting Libraries & Tools:

- **Python 3** — main programming language used for automation and integrating reconnaissance tools.
- **requests** — Python library used for sending HTTP requests during vulnerability testing.
- **json** — used for processing and analyzing scan results.
- **subprocess** — used to execute external tools such as Nmap, Subfinder, and Dirsearch from Python scripts.

Result Collection / Visualization:

- **Streamlit** — used to create a web-based dashboard where users can enter a target domain and view reconnaissance results.
- **JSON output files** — used to store scan results in structured format for analysis.

Programming & Scripting:

- **Python 3 with libraries such as Streamlit, Requests, JSON, and Scapy** for automation, scanning integration, and AI-assisted vulnerability analysis.
- **Bash / Command Prompt utilities** for executing reconnaissance tools and managing configurations.

Other Useful Tools:

- **Git** — for downloading and managing open-source reconnaissance tools.
- **Browser (Chrome/Firefox)** — for accessing the Streamlit dashboard interface.
- **Terminal / Command Prompt** — for running scanning tools and project scripts.

Software Requirements:

Operating System

Windows 10 / Windows 11 used as the primary operating system for developing and running the project.

Development Environment

Visual Studio Code (VS Code) used as the main code editor for writing and managing the project code.

Python 3 used as the primary programming language for implementing the reconnaissance and vulnerability testing system.

Reconnaissance and Scanning Tools

Subfinder — used to discover subdomains of the target domain.

Nmap — used for port scanning and identifying open services.

WhatWeb — used to detect web technologies such as CMS, frameworks, and servers.

Dirsearch — used to find hidden directories and files on web servers.

Python Libraries

Requests — used for sending HTTP requests to target systems.

JSON — used for storing and processing scan results.

OS and Subprocess — used to execute external scanning tools from Python.

Visualization and Interface

Streamlit — used to create a simple graphical interface and dashboard for displaying scan results and analysis.

Version Control

Git — used for managing project files and maintaining version control.

htop / Task Manager — used for monitoring system performance during scans.

Software Requirement Analysis (SRA) defines the software components, tools, and configurations necessary to implement the **Reconnaissance and Vulnerability Tester Using AI** using Python and various reconnaissance tools within a Windows or Linux environment.

1. Operating System

- **Windows 10 / Windows 11 / Linux (Ubuntu recommended)**

- o Acts as the main operating system for developing and running the reconnaissance and vulnerability testing framework.
- o Provides the required environment for installing Python, reconnaissance tools, and supporting libraries used in the project.

2. Development Platform

- **Visual Studio Code (VS Code)**

- o Used as the primary development environment for writing and managing Python scripts.
- o Provides extensions for Python development, debugging, and project management.

3. Reconnaissance & Vulnerability Scanning Tools

- **Subfinder (latest version)**

- o Core tool used for discovering subdomains of the target domain during the reconnaissance phase.
- o Helps identify additional attack surfaces such as hidden services, APIs, and administrative portals.

- **Nmap (latest stable release)**

- o Core software used for scanning open ports and identifying services running on the target system.
- o Features include:

- Port scanning and service detection
- Identification of open and filtered ports
- Network service enumeration
- Detection of potential security risks

- **WhatWeb**

- o Used for identifying technologies used by the target website.
- o Detects web servers, content management systems, programming languages, and web frameworks.

- **Dirsearch**

- o Used for discovering hidden directories and files on the target web server.
- o Helps identify sensitive directories such as:

- /admin
- /login
- /backup
- /uploads

4. Supporting Libraries & Dependencies

- **Python 3** → Required for automation and integration of reconnaissance tools.
- **requests** → Used for sending HTTP requests to target websites during testing.
- **json** → Used for processing and analyzing scan results.
- **subprocess** → Allows execution of external tools such as Nmap, Subfinder, and Dirsearch from Python scripts.

5. Network & Web Analysis Tools

- **Nmap** → Used for scanning open ports and identifying services on the target system.
- **Subfinder** → Used for subdomain enumeration during reconnaissance.
- **WhatWeb** → Used for detecting technologies used by the target website.
- **Dirsearch** → Used for discovering hidden directories and files on web servers.

6. Result Management & Visualization (Optional but Recommended)

- **Streamlit** → Used to create a web-based dashboard interface for running scans and viewing results.
- **JSON Output Files** → Used to store scan results in structured format for analysis and reporting.
- **Web Browser (Chrome / Firefox)** → Used to access the Streamlit dashboard and visualize the scanning results.

Software Design

The software design phase defines the architecture, modules, and interaction flow of the **Reconnaissance and Vulnerability Tester Using AI**. It explains how different reconnaissance tools integrate with Python to discover domains, scan open ports, detect technologies, and identify potential vulnerabilities in target systems.

Furthermore, this phase maps out the end-to-end data pipeline, detailing how raw scan results from various subsystems are aggregated, parsed, and normalized into a unified format. By establishing clear data structures, the design ensures that the AI engine can seamlessly ingest this structured telemetry. The AI module then applies machine learning models to correlate disparate data points, filter out false positives, and prioritize vulnerabilities based on their potential impact and exploitability.

SYSTEM ARCHITECTURE

The system follows a layered architecture with the following components:

1. Data Input Layer

- o Responsible for receiving the target domain or website URL from the user.
- o The user enters the domain through the **Streamlit dashboard interface**.
- o Front-End Input Validation Mechanism

2. Reconnaissance Engine

- o Performs automated reconnaissance on the target domain.
- o Integrates multiple security tools such as **Subfinder, Nmap, WhatWeb, and Dirsearch**.
- o Discovers subdomains, scans open ports, detects technologies used by websites, and finds hidden directories.

3. AI Analysis Layer

- o Processes the results generated by reconnaissance tools.
- o Analyzes scan data to identify potential vulnerabilities or security risks.
- o Detects suspicious configurations such as exposed ports, outdated technologies, or sensitive directories.

4. Result Logging Layer

- o Stores scanning results in structured formats such as **JSON files**.
- o Maintains records of discovered subdomains, open ports, technologies, and directories.

5. Visualization & Reporting Layer

- o Uses **Streamlit dashboard** to display results to the user.
- o Provides an easy-to-understand interface for viewing reconnaissance results and vulnerability analysis.

6. Management Layer

- o Responsible for managing scanning tools and configurations.
- o Allows administrators to update reconnaissance tools and manage system settings.

1. System Flow (Data Flow Diagram – Textual Description)

User Input → The user enters a target domain in the Streamlit dashboard.

The operational life cycle of the framework begins at the boundary of the client interface.

The system remains in an idle state, running a continuous event loop via Streamlit, until an administrative operator enters a target string into the primary text container on the web control panel.

This interaction serves as the critical gatekeeper for the downstream automation pipeline, executing several hidden programmatic validations before any background process is triggered:

- **String Interception and Ingestion:** The moment a user inputs a target value (such as `example.com`, `https://sub.target.org/index.php`, or a raw IPv4 address like `192.168.1.1`) and commits the action, the front-end state manager catches the string payload. Streamlit immediately binds this input to a session state variable (`st.session_state.target_domain`), ensuring the value remains persistent across subsequent page reruns.
- **Interface State Locking:** To maintain strict application stability and prevent resource race conditions, the input module dynamically modifies the frontend UI state upon submission. It instantly disables the input text box and the "Initiate Scan" button. This locking mechanism prevents the user from accidentally clicking the submission trigger multiple times, which would otherwise spawn duplicate, conflicting operating system sub-processes and overwhelm the host server's CPU.
- **Visual Feedback Layer Initialization:** Simultaneously, the input module clears the previous dashboard viewports and launches animated rendering threads. It displays a real-time progress ticker, visual loading indicators, and informational status alerts. This provides the operator with instant, transparent feedback while the backend environment sets up the execution workspace.

- **Automated URL Sanitization and Parsing:** Raw inputs from security analysts can be messy and inconsistent. The input module leverages Python's native string manipulation utilities to normalize the text payload into a strict format required by command-line tools. It automatically strips away trailing directory slashes, accidental whitespace characters, and application-layer protocols (such as `http://` or `https://`), isolating the bare, clean root domain or network host target.
- **Front-End Input Validation Regular Expressions (Regex):** Before passing any string down to the operating system shell, the input layer acts as a strict validator. It runs the sanitized string against a predefined network syntax regex matrix. This step verifies that the input complies with official RFC domain standards or standard IP formatting rules, immediately throwing a clear visual error on the dashboard if the input contains illegitimate characters or broken structures.
- **Defensive Shell-Injection Mitigation:** In security automation tools, passing user inputs directly to command-line utilities presents a catastrophic vulnerability risk known as OS Command Injection. If a malicious user inputs a domain appended with shell metacharacters (e.g., `target.com; rm -rf /` or `target.com && whoami`), a naive backend would execute the injected commands with system privileges. The User Input Module completely neutralizes this attack vector by isolating the input inside an explicit string token array before passing it to Python's execution engine, ensuring the entire input is treated strictly as a literal domain name string rather than executable code.
- **Target Availability Pre-Flight Pings:** To maximize processing efficiency and eliminate wasted execution time, the input module performs a rapid network connectivity check before launching resource-heavy local binaries. It utilizes low-overhead web requests to check if the target domain actively resolves to an online public IP address. If the host fails to respond within a strict timeout window (e.g., 3 seconds), the framework halts the execution loop, releases the interface locks, and displays an error alert stating that the target system is currently offline or unreachable.

Reconnaissance Execution → The system runs reconnaissance tools such as Subfinder, Nmap, WhatWeb, and Dirsearch on the target domain.

Data Collection → Results such as discovered subdomains, open ports, technologies, and directories are collected. the target input string passes all pre-flight sanitization and availability checks, the framework transitions from an idle posture into an active auditing state. The Reconnaissance Execution Module acts as the primary engineering

workforce of the platform, bridging the gap between Python's high-level runtime and the lower-level operating system shell.

Instead of relying on basic script commands that execute sequentially, this module handles parallel process orchestration, asynchronous data streaming, and resource balancing.

AI Analysis → The system analyzes the collected data to detect possible vulnerabilities and security risks.

The transition from data collection to cognitive evaluation represents the primary analytical milestone of the framework. Once the asynchronous execution layer confirms the successful termination of the reconnaissance tools, the raw, isolated datasets are synthesized into a single, unified data profile. This profile is securely transmitted to the AI Analysis Module, which functions as the primary logical reasoning engine of the platform.

Instead of performing rigid, linear lookups, the AI analysis phase introduces a dynamic, context-aware layer of intelligence that mimics the analytical methodologies of a human penetration testing expert.

To understand the operational necessity of the AI Analysis layer, it is essential to contrast it with traditional automated vulnerability assessment engines:

- **The Rigid Failure of Static Rule Bases:** Traditional scanners rely entirely on signature matching. They scan a target, extract a version string, and check it against a static database of known vulnerabilities. If a software version is slightly modified or if a vulnerability relies on a complex chain of multiple minor configurations, traditional signature scanners fail to recognize the threat, leaving critical security gaps completely unflagged.
- **Contextual Intelligence and Heuristic Synthesis:** The AI Analysis Module goes beyond simple string matching by evaluating the entire attack surface simultaneously. It reads the collective output of the reconnaissance cluster as an interconnected ecosystem. This allows the system to evaluate how a minor, informational finding from one tool (such as an exposed directory path discovered by Dirsearch) directly impacts or escalates a vulnerability found by another tool (such as an open port cataloged by Nmap).

The AI Analysis Module processes the unified reconnaissance data through a structured pipeline to ensure technical accuracy, consistency, and contextual relevance:

- **Dynamic Ingestion and System Prompt Engineering:** The system maps the parsed data payload into a highly specialized system prompt template. This prompt establishes strict operational boundaries for the artificial intelligence model. It commands the model to act exclusively as an elite offensive security engineer and defensive solutions architect, framing its data interpretation through an objective, highly analytical lens.
- **Enforcement of Deterministic Output Configurations:** To eliminate the risk of neural network

hallucinations—where an AI might invent non-existent CVE numbers or recommend broken, imaginary code patches—the module establishes strict API runtime parameters. It enforces a temperature setting of exactly 0.0. This suppresses creative variance within the model's token selection process, ensuring that every generated security insight is entirely deterministic, repeatable, and technically accurate across multiple evaluation cycles.

- **Asynchronous API Gateway Routing:** The system transmits the structured target profile via a secure, encrypted TLS connection to the remote Large Language Model endpoint. The module utilizes non-blocking network calls to prevent the Streamlit user interface from locking up during the thinking phase, allowing the dashboard to remain responsive while the AI processes complex reasoning chains.

The true core innovation of the AI module is its ability to perform automated vulnerability chaining. When analyzing a target domain, the AI logic engine actively hunts for logical correlations across distinct computing layers:

- **Cross-Layer Risk Correlation:** If Nmap reports an open alternate port, WhatWeb reports that the port runs a specific out-of-date web framework, and Dirsearch reports a 200 OK status for a hidden administrative backup file on that same path, the AI module synthesizes these findings into a unified attack narrative. It recognizes that the exposed backup file likely contains configuration credentials that can be used to authenticate into the outdated framework, creating a high-probability exploit path.
- **Discovery of Complex Access Paths:** By matching separate technical indicators together, the AI deduces advanced threat vectors—such as how an unencrypted HTTP transmission framework combined with a specific login panel path could allow an adversary to intercept administrative sessions via a man-in-the-middle attack.

Result Generation → The system generates results showing whether the domain is secure or potentially vulnerable.

The generation of results serves as the definitive analytical output stage of the framework, translating complex multi-tool datasets and AI reasoning chains into a highly structured, actionable security posture intelligence report. Once the remote Large Language Model completes its deterministic processing loop, the application captures the raw, text-based intelligence stream and transforms it into an objective, data-driven security evaluation.

This phase is mathematically and logically designed to give administrators a definitive, binary understanding of their perimeter health—**Secure vs. Potentially Vulnerable**—while providing the

granular evidence needed to support that conclusion.

Result Storage → Scan results are stored in JSON or text format for future reference.

The local data logging and storage phase serves as the critical persistence architecture of the platform. Once the result generation engine completes the assembly of the security posture reports, the framework shifts execution control to the storage backend. In an enterprise security application, letting data exist purely in transient runtime memory or volatile front-end dashboard states is a major engineering risk. A single browser refresh or network timeout would instantly destroy hours of complex, resource-heavy scanning data.

To prevent data loss and establish an immutable audit trail, this module serializes and writes both the raw tool execution logs and the cognitive AI threat assessments directly onto the host system's non-volatile physical storage array.

Visualization → The results are displayed on the Streamlit dashboard for the user to review.

The visual dashboard rendering phase represents the final layer of the application's core data pipeline.

After the local storage module successfully writes the structured JSON and plain-text evaluation files to disk, control is handed back to the user interface.

In cybersecurity tooling, data clarity is just as critical as data collection. If a platform simply outputs massive blocks of raw JSON data or chaotic text strings, security analysts will waste valuable time trying to locate the highest-priority issues. The Visualization Module utilizes **Streamlit**, a reactive python web framework, to convert these complex datasets into an interactive web interface.

2.Module Design

- **User Input Module** → Accepts the target domain from the user through the dashboard.
- **Subdomain Discovery Module** → Uses **Subfinder** to identify subdomains related to the target domain.
- **Port Scanning Module** → Uses **Nmap** to detect open ports and running services.
- **Technology Detection Module** → Uses **WhatWeb** to identify technologies used by the website.
- **Directory Discovery Module** → Uses **Dirsearch** to find hidden directories and files.
- **AI Analysis Module** → Analyzes reconnaissance results and identifies possible vulnerabilities.
- **Logging & Reporting Module** → Stores and displays scanning results for analysis.

2. Design Considerations

- **Performance:** The system automates reconnaissance tools to reduce manual effort and improve scanning efficiency.
- **Scalability:** The system can scan multiple domains and store results for future analysis.
- **Flexibility:** The framework supports integration of additional security tools in the future.
- **Usability:** The Streamlit dashboard provides a simple interface for launching scans and viewing results.

2.1 Diagrams

1. Level 0: Context Diagram (High-Level View)

This is the most basic view of your system. It is used to show the absolute boundaries of the software by treating the entire system as a single "black box."

- **The Actor (User):** A security analyst or penetration tester inputs data (like a target domain name, e.g., `example.com`) into the system.
- **The System Block:** The entire platform is represented as a single blue rectangle (*Reconnaissance and Vulnerability Testing System*). None of the internal tools or code are visible here.
- **The Output (Scan Results):** The system processes the request and sends out the final payload, which is a structured vulnerability report or dashboard view for the user.

2. Level 1: DFD (Detailed Sub-Processes)

This level breaks open the "black box" from Level 0 to show the **five major functional pillars** inside your software architecture and how they hand off data to one another:

- **User Input Module:** Takes the domain or IP address from the user, cleans it up (sanitizes it to ensure it is safe), and sets up the configuration.
- **Reconnaissance Tools Execution:** The central engine (likely written in Python) that actually runs the security scanners.
- **AI Analysis Module:** Receives the raw scan data, figures out which vulnerabilities are real, removes false positives, and assesses the risk.
- **Result Logging:** The database storage layer (like PostgreSQL or MongoDB) where all raw and processed data is saved.
- **Visualization Dashboard:** The user interface front-end that reads from the logging database to display charts, graphs, and metrics to the user.

3. Level 2: Detailed DFD (In-Depth Module Breakdown)

This level takes the "**Reconnaissance Tools Execution**" block from Level 1 and zooms in completely. It shows that your scanning engine is actually a **sequential pipeline** where 4 major security tools run one after another, with each tool handing its output to the next:

1. **Subdomain Discovery (Subfinder):** Takes the root domain (e.g., `target.com`) and discovers hidden subdomains (e.g., `admin.target.com`, `dev.target.com`).
2. **Port Scanning (Nmap):** Takes those subdomains, finds their active IP addresses, and checks which network ports are wide open.
3. **Technology Detection (WhatWeb):** Looks at the open web ports found by Nmap and discovers what software stack they are running (like WordPress, Apache, or PHP).
4. **Directory Discovery (Dirsearch):** Brute-forces the web URLs to find hidden files and folders (like `/backup.zip` or `/admin/login`).
5. **AI Analysis & Reporting:** Gathers all the collected pieces (subdomains, ports, software versions, hidden paths) and uses an AI model to evaluate the final threat level.

Summary

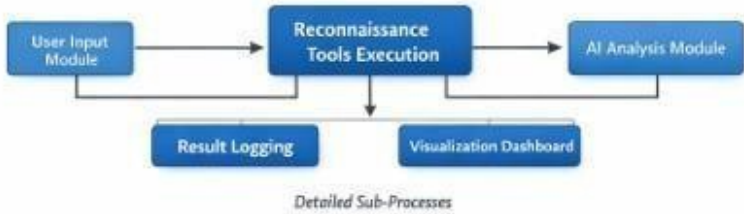
- **Level 0** → Entire system shown as one process (input and output only).
- **Level 1** → Major modules such as reconnaissance, analysis, and reporting.
- **Level 2** → Detailed breakdown of reconnaissance tools and AI analysis components.

Data Flow Diagram (DFD)

Level 0: Context Diagram



Level 1: DFD



Level 2: Detailed DFD



E-R Diagram (Entity-Relationship Diagram)

Explanation of Entities & Relationships

- **User** → The person who enters the target domain or website to perform reconnaissance and vulnerability testing.
- **Target Domain** → The website or domain that is scanned by the system for reconnaissance and vulnerability analysis.
- **Reconnaissance Tools** → Security tools such as **Subfinder**, **Nmap**, **WhatWeb**, and **Dirsearch** used to collect information about the target domain.
- **AI Analysis Module** → The component that analyzes the results generated by reconnaissance tools and identifies possible vulnerabilities or security risks.
- **Scan Results** → The output generated after scanning, including discovered subdomains, open ports, detected technologies, and hidden directories.
- **Logs / Reports** → Records of the scanning process and vulnerability analysis stored in structured formats such as JSON or text files.

Relationships

- **User** provides the **Target Domain** for scanning.
- **Reconnaissance Tools** analyze the **Target Domain** to gather information.
- The **AI Analysis Module** processes the results generated by the reconnaissance tools.
- The system produces **Scan Results** and stores them as **Logs / Reports**.
- **User / Administrator** reviews the scan results and reports to understand the security status of the domain.

- Structural Decomposition of System Entities

To thoroughly understand the data layout of the **Reconnaissance and Vulnerability Tester Using AI**, we must look past basic definitions and examine the specific technical traits, database fields (attributes), and operational constraints linked to each.

1.3.1 The User / Administrator Entity

The system classifies human actors based on permissions and operational roles. In production environments, this entity is managed using Role-Based Access Control (RBAC):

- **Attributes:** User_ID (Primary Key), Username, Password_Hash (SHA-256/Bcrypt), Email_Address, Role_Type (Standard Analyst vs. System Admin), and Last_Login_Timestamp.
- **Operational Constraints:** A single user can run multiple distinct scanning jobs, but they must have a valid security token to make calls to the backend tools module.

1.3.2 The Target Domain Entity

This entity represents the target profile provided by the analyst. It acts as the core anchor point for all collected intelligence data.

- **Attributes:** Domain_ID (Primary Key), Fully_Qualified_Domain_Name (FQDN), Root_IP_Address, Organization_Owner, Scope_Status (In-Scope / Out-of-Scope validation flag), and Discovery_Date.
- **Data Integrity Rules:** The system enforces strict string formatting constraints. Duplicate entries of identical active domains are blocked or grouped together under a single master profile to prevent redundant scanning behavior.

1.3.3 The Reconnaissance Tools Entity

This entity represents the underlying technical toolkit container (Subfinder, Nmap, WhatWeb, and Dirsearch). It acts as a bridge that translates generic user goals into specific command-line parameters.

- **Attributes:** Tool_ID (Primary Key), Tool_Name, Binary_Path, Current_Version, Execution_Arguments (dynamic CLI flags), and Execution_Status (Idle, Running, Errored).

1.3.4 The AI Analysis Module Entity

This entity manages the advanced logic models that process the data streams. It functions as an analytical service container rather than a static table.

- **Attributes:** Model_ID (Primary Key), Algorithm_Type (Classification/NLP), Model_Weight_Version, Confidence_Threshold_Setting, and Last_Training_Date.

1.3.5 The Scan Results Entity

This is a highly dynamic data block that holds the raw, unpolished text and telemetry findings returned directly from network interactions.

- **Attributes:** Result_ID (Primary Key), Scan_Job_ID (Foreign Key), Discovered_Subdomains_List, Open_Ports_Vector, Identified_Tech_Stack, and Hidden_Paths_Array.

1.3.6 The Logs / Reports Entity

This entity represents the finalized data output. It holds the polished threat intelligence summaries generated after the AI finishes filtering the results.

- **Attributes:** Report_ID (Primary Key), Result_ID (Foreign Key), Vulnerability_Count, Overall_Risk_Rating (Critical, High, Medium, Low), File_Path_Location (JSON/PDF destinations), and Generation_Timestamp.

1.4 Deep-Dive Analysis of Relationships and Cardinality

- **Cardinality:** One-to-Many (\$1:N\$).
- **Technical Logic:** A single security analyst can enter many different target domains into the portal over time. Conversely, a specific target domain scan record is initiated by exactly one authorized user.
- **Database Rule:** The `User_ID` value is injected directly into the target record table as a Foreign Key, ensuring clear accountability for every initiated scan.

1.4.2 Reconnaissance Tools "Process / Analyze" Target Domain (N:M Relationship)

- **Cardinality:** Many-to-Many (\$N:M\$).
- **Technical Logic:** At the same time, a single tool instance can be assigned to scan multiple domains simultaneously within an asynchronous queue.
- **Database Rule:** This complex relationship is resolved in the database using an intermediate junction table named `Tool_Executions`. This table tracks specific operational context metrics, including execution start times, memory utilization, and process return codes:

Execution_ID (PK)	Scan_Job_ID (FK)	Tool_ID (FK)	Domain_ID (FK)	Run_Status
EX-9081	SC-4412	TL-02 (Nmap)	DM-003	Completed
EX-9082	SC-4412	TL-04 (Dirsearch)	DM-003	Active

1.4.3 AI Analysis Module "Evaluates" Reconnaissance Output (1:N Relationship)

- **Cardinality:** One-to-Many (\$1:N\$).
- **Technical Logic:** It cleans and structuralizes these messy data points into organized entries within the `Scan Results` database table.

1.4.4 Scan Results "Stored As" Logs / Reports (1:1 Relationship)

- **Cardinality:** One-to-One (\$1:1\$).
- **Technical Logic:** Every individual scan job results in exactly one definitive historical log record and remediation report. This clean 1:1 pairing prevents data mixing, ensuring that security analysts can trace findings directly back to the specific execution run for verification.

1.5 System Data Lifecycle and Operational Workflow

By mapping the ER diagram to an active runtime environment, we can trace the lifecycle of system data through four distinct, operational phases:

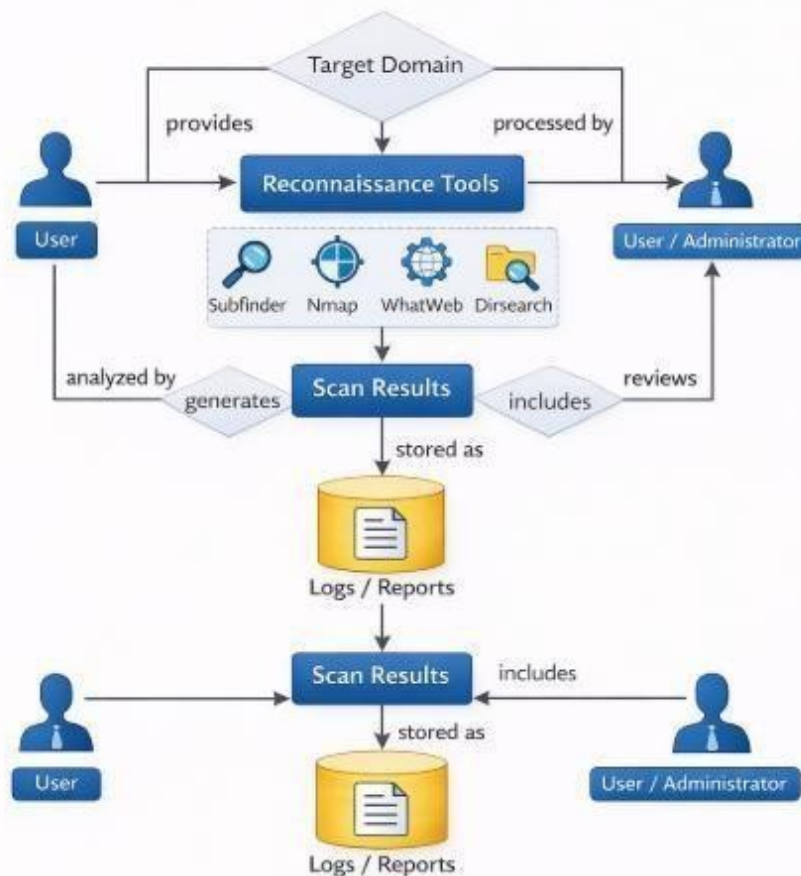
1.The Ingestion Phase: The User authentication token is validated at the interface boundary. Once approved, the user creates a Target Domain record, setting the overall scope boundaries for the automation engine.

2.The Orchestration Phase: The system activates the Reconnaissance Tools container. The engine spins up Subfinder to gather subdomains, passes those hosts to Nmap to detect open network ports, runs WhatWeb to fingerprint active technology stacks, and uses Dirsearch to find hidden paths.

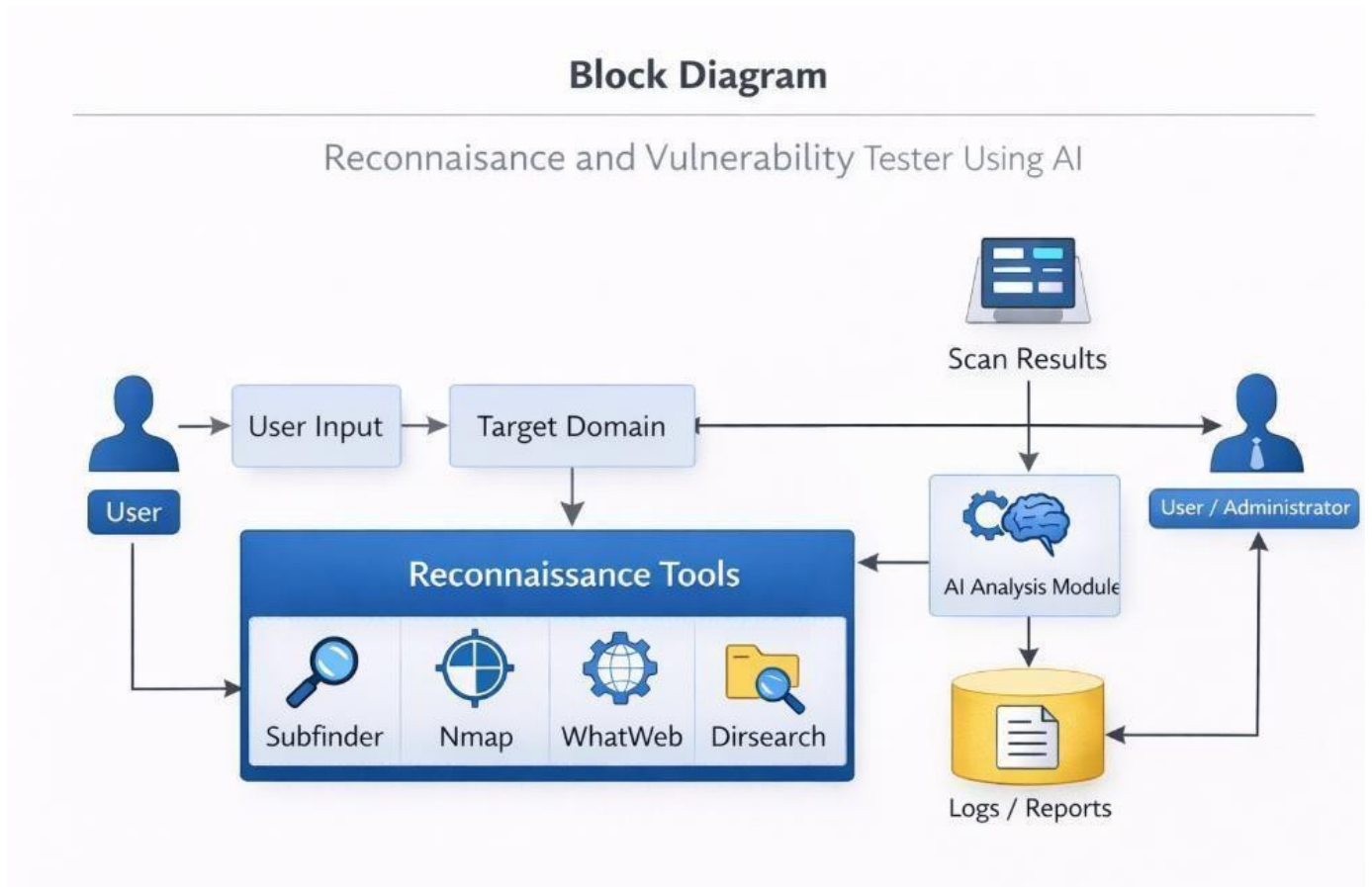
3.The Intelligence Phase: The raw outputs from the scanning tools are fed into the AI Analysis Module. The AI runs correlation algorithms to cross-reference software versions against live exploit databases, filters out false alerts, and flags high-priority security concerns.

4.The Persistence & Review Phase: The refined intelligence dataset is saved into the database as a Scan Result entry and exported as an immutable Logs / Reports document. The User / Administrator then views this data on the dashboard to quickly identify critical flaws and deploy fixes.

1. E-R Diagram (Entity-Relationship Diagram)



4.4.1 Block Diagram:



4.5 AI-Driven Security Testing and Vulnerability Analysis

4.5.1 Introduction to AI-Driven Security Testing

Artificial Intelligence (AI)-driven security testing is an advanced cybersecurity approach used to automate reconnaissance and vulnerability analysis of target systems. Traditional security testing methods require cybersecurity professionals to manually execute multiple reconnaissance tools and analyze the results separately. This process can be time-consuming, complex, and prone to human error.

The proposed Reconnaissance and Vulnerability Tester Using AI combines automated reconnaissance tools with AI-assisted analysis to identify vulnerabilities and determine whether a domain is secure or vulnerable. The system automatically performs subdomain discovery, port scanning, technology detection, and hidden directory discovery using integrated reconnaissance tools.

The AI analysis module processes the collected reconnaissance data and identifies possible security risks such as exposed services, vulnerable technologies, and hidden administrative directories. The system helps users understand the attack surface of a target domain and provides intelligent insights regarding potential vulnerabilities.

AI-driven security testing improves the efficiency of vulnerability assessment by reducing manual effort and increasing the speed of reconnaissance analysis. The framework also provides centralized visualization through the Streamlit dashboard, making it easier for users to analyze the collected security information.

To appreciate the necessity of an AI-driven approach, one must look at the operational bottlenecks that plague traditional security workflows.

Conventional vulnerability management relies heavily on isolated tools that do not communicate with one another. A human analyst must manually copy the text output from a subdomain discovery tool, clean it using custom scripts, feed it into a port scanner, and then manually cross-reference the discovered software versions against static vulnerability databases like the National Vulnerability Database (NVD).

This traditional approach introduces three major failure points:

- **Context Fragmentation:** Information discovered at the beginning of a scan (such as a specific naming convention in a subdomain) is lost by the time the analyst looks at a directory brute-force report.

- **Alert Fatigue:** Standard security tools flag thousands of theoretical vulnerabilities based purely on version numbers, forcing analysts to spend hours filtering out harmless false positives.
- **Delayed Time-to-Mitigation:** By the time a comprehensive manual scan is completed, analyzed, and written into a report, an attacker may have already exploited the exposed vulnerability.

The Reconnaissance and Vulnerability Tester Using AI ,eliminates these gaps by wrapping the entire pipeline into a single, unified execution loop. It treats security testing not as a series of separate tasks, but as an interconnected data pipeline where information flows smoothly from collection to final intelligence.

4.5.3 Architectural Synthesis of the AI-Assisted Core

The underlying intelligence of this framework does not arise from simply running the integrated reconnaissance tools in a parallel script, but rather from how the system synthesizes, structurally transforms, and logically correlates their independent outputs.

When raw data leaves the isolated tool execution layer, it undergoes a multi-phased transformation process before it ever reaches the Artificial Intelligence reasoning module. This architectural core acts as an automated security refinery, transforming chaotic terminal outputs into structured, context-rich security intelligence.

1. Data Aggregation and Structural Normalization

The raw text arrays and terminal output strings generated by the various utilities are inherently volatile, unstructured, and written in completely different formats. For example, `Nmap` outputs plain-text command tables, `Subfinder` returns flat newline-separated string blocks, `WhatWeb` generates dense token streams packed with application meta-tags, and `Dirsearch` builds real-time execution logs with specific HTTP response codes.

To bridge this data communication gap, the platform implements a dedicated parsing and ETL (Extract, Transform, Load) engine.

1. **The Serialization Mechanism:** The system utilizes custom regular expression (Regex) matrices and structural string parsers to strip away terminal ANSI color codes, metadata fluff, and command execution telemetry. It isolates the bare diagnostic data points—such as explicit integer port profiles, application layer version strings, root domain associations, and endpoint status codes.
2. **Unified Target JSON Schema:** These parsed data points are serialized into a highly structured, centralized

JSON database model. This uniform schema collapses the boundaries between network-layer statistics and web-application architectures, binding all discoveries into a single object model.

2. Contextual Correlation and Vulnerability Chaining

The AI module behaves as an automated security architect and cognitive threat modeler. Traditional automated vulnerability assessment systems suffer from a fatal design flaw: they analyze security vulnerabilities in complete isolation. A standard scanner evaluates an open network port without looking at the directory traversal logs, missing the bigger picture.

Our framework's AI core applies **Chain-of-Thought (CoT)** reasoning models to evaluate the target domain collectively, evaluating the data points simultaneously to identify complex exploit chains.

- **The Convergence of Isolated Indicators:** In this scenario, `Nmap` identifies an open alternate HTTPS port (8443). Simultaneously, `WhatWeb` inspects the response headers of that port and fingerprints an outdated Java/Spring Boot microservice framework. At the same time, the `Dirsearch` worker reveals a publicly accessible archive at `/backup/config.zip` returning a 200 OK HTTP status code.

- **Intelligent Threat Inference:** A legacy scanner would log these as three independent, medium-risk findings. The AI module, however, synthesizes these findings into a unified attack narrative. It reasons that an attacker could download the exposed `/backup/config.zip` file, extract hardcoded database credentials or encryption keys, and leverage known remote code execution vulnerabilities within the outdated Java framework running on port 8443 to completely compromise the underlying server. By connecting these dots, the framework flags a critical exploitation path that would otherwise go unprioritized.

Dynamic Risk Categorization and Environmental Scoring

A recurring problem within enterprise cybersecurity operations is "alert fatigue"—a phenomenon where security teams are overwhelmed by thousands of high-severity alerts generated by rigid scanning tools, making it impossible to determine what to patch first.

Instead of relying entirely on static, rigid **CVSS (Common Vulnerability Scoring System)** metrics which evaluate bugs based solely on theoretical severity, our AI core implements a dynamic, context-aware risk engine.

The AI automatically evaluates the business posture, exposure vector, and architectural landscape of the discovery

to re-prioritize severity rankings in real time:

- **Exposure and Asset Criticality Context:** An open, unauthenticated administrative path or development panel found on a forgotten, unmonitored staging subdomain (`dev.targetsite.org`) is automatically elevated to a higher priority risk category than the same exact path on a heavily monitored, locked-down production server. The AI understands that development environments typically lack robust intrusion detection systems (IDS) and are frequently used by developers to store unencrypted database mirrors, making them a prime target for initial access.
- **Intelligent Remediation Allocation:** If an outdated technology component is identified but the data inputs prove that the component is fully shielded behind a modern cloud proxy or returns a secure access denied state (`403 Forbidden`), the AI dynamically scales down the risk priority. This environmental adjustments ensure that the final generated security report contains an accurate, localized threat map, allowing developers to safely skip the noise and immediately fix the flaws that present the most immediate real-world danger.

4.5.4 Strategic Benefits of the Streamlit Presentation Layer

A security tool is only as effective as its ability to clearly communicate findings to the person using it. The integration of a centralized **Streamlit dashboard** transforms complex, nested command-line data into an intuitive, interactive visual space. This presentation layer delivers several key advantages:

- **Real-Time Pipeline Tracking:** Analysts can watch data move through the system in real time, tracking live progress as the backend cycles from subdomain discovery through to directory brute-forcing.
- **Interactive Attack Surface Mapping:** The front-end organizes data into clean, filterable tables and charts, allowing users to quickly sort assets by open ports, active technologies, or risk levels.
- **Actionable Remediation Delivery:** Instead of presenting users with a raw list of code flaws, the dashboard displays clear, prioritized remediation steps. This bridges the gap between discovering a security loophole and executing a fix.

4.5.5 Practical Applications and Strategic Impact

The scalable design of this AI-driven system makes it highly effective across a variety of modern security environments:

- **Continuous Attack Surface Management (CASM):** Organizations can run this tool on a regular schedule to map their public internet footprint, instantly flagging newly exposed ports or unauthorized shadow IT setups.
- **Proactive Penetration Testing:** Security teams can use the framework to simulate the initial gathering phases of an actual cyberattack, revealing exactly what an adversary would see during a real-world exploit attempt.
- **DevSecOps Pipeline Ingestion:** The lightweight Python architecture allows the system to easily plug into CI/CD deployment pipelines, scanning new staging environments for vulnerabilities before they are pushed live to the public.

By shifting the burden of data gathering and false-positive filtering from the human analyst to an automated, intelligent system, this architecture sets a new standard for modern security assessments. It ensures that security teams can stop spending time managing individual tools and start focusing their energy on fixing verified vulnerabilities.

4.5.2 Working of AI-Based Vulnerability Analysis

The AI-driven vulnerability analysis process consists of multiple stages that work together to collect reconnaissance data, analyze vulnerabilities, and generate security insights.

1. User Input Stage

The process begins when the user enters a target domain or website URL into the Streamlit dashboard interface. The target domain acts as the primary input for the reconnaissance and vulnerability testing process.

Example:

- google.com
- example.com
- targetsite.org

The system validates the entered domain before initiating the reconnaissance phase.

2. Reconnaissance Execution Stage

After validation, the system automatically executes multiple reconnaissance tools

including:

- Subfinder
- Nmap
- WhatWeb
- Dirsearch

These tools collect different types of security-related information about the target domain.

3. Data Collection Stage

The reconnaissance tools generate results such as:

- Discovered subdomains
- Open ports
- Running services
- Technologies used
- Hidden directories and files

The collected results are stored in structured JSON format for further analysis.

4.Data Processing & Structuring Stage

Before raw data can be passed to an Artificial Intelligence model, it must be cleaned, normalized, and structured.

Raw text output from tools like Nmap, Subfinder, or Dirsearch is inherently chaotic, containing redundant whitespace, human-readable filler text, and varying data formats (such as XML, JSON, or plain text).

The data processing layer acts as a pipeline that refines this raw information:

- **Parsing and Extraction:** The system uses regular expressions (Regex) and specialized data parsers to strip out irrelevant terminal styling and metadata. It extracts only the essential security data points, such as IP addresses, specific port numbers, service banners, software version strings, and HTTP status codes.
- **JSON Serialization:** The extracted data points are structured into a standardized, machine-readable JSON object. This creates a uniform schema representing the target's entire attack surface, making it easy for the downstream AI engine to digest.
- **Contextual Tagging:** The pipeline enriches the data by adding contextual metadata flags (e.g., classifying a discovered service as a "Database Interface," "Web Server," or "Outdated Legacy Protocol").

5. AI Analysis & Vulnerability Mapping Stage

This stage is the core intelligence engine of the system. Once the structured JSON data is ready, it is fed into an advanced Large Language Model (LLM) or a specialized Security Machine Learning engine. The AI acts as an automated expert penetration tester, evaluating the collected data through several logical layers:

Cross-Referencing Knowledge Bases

The AI engine does not rely solely on static rule sets. It dynamically evaluates the open ports, services, and software versions against massive, vector-embedded security databases. It looks for correlations with known **CVEs (Common Vulnerabilities and Exposures)**, zero-day threat patterns, and common misconfigurations associated with the specific technologies identified by WhatWeb.

Reasoning and Heuristic Evaluation

Unlike traditional signature-based scanners that only flag exact matches, the AI applies contextual reasoning:

- **Chaining Anomalies:** The AI evaluates whether multiple low-severity findings can be linked together to form a critical exploit path. For example, if Dirsearch finds a hidden `/backup` directory and Nmap reveals an open SMB port, the AI reasons that an attacker might try to look for leaked credentials to access that port.
- **False Positive Reduction:** The model analyzes HTTP response patterns and service headers to filter out "honeypots" or misleading version banners, ensuring that only plausible, high-probability risks are pushed forward.

6. Security Insights & Report Generation Stage

The final stage translates complex, technical AI findings into a comprehensive, actionable security report designed for both system administrators and developers. The AI dynamically compiles the analysis into structured sections within the Streamlit dashboard:

Executive Summary

A high-level, human-readable overview of the target's security posture. It highlights the overall risk level (e.g., Low, Medium, High, Critical) and summarizes the most urgent threats discovered during the cycle, giving stakeholders an immediate understanding of their exposure.

Detailed Vulnerability Breakdown

For every identified risk, the system generates a dedicated profile containing:

- **Vulnerability Name & Description:** A clear explanation of what the vulnerability is and how it functions.
-
- **Impact Assessment:** An evaluation of what an attacker could achieve if they successfully exploited the flaw (e.g., Remote Code Execution, Data Leakage, Denial of Service).
- **Severity Rating:** A calculated risk score based on accessibility and potential damage.

Actionable Remediation Guidance

The AI provides step-by-step mitigation instructions tailored to the specific environment. Instead of generic advice, it outputs exact remediation steps, which may include:

- The precise commands required to update an outdated service.
- Specific configuration changes needed to close an open port safely.
- Code snippets or firewall rules to block unauthorized directory traversal.

4. AI Analysis Stage

The AI analysis module processes the collected reconnaissance data and identifies possible vulnerabilities and security risks. The AI module examines:

- Open ports and exposed services
- Technologies with known vulnerabilities
- Sensitive directories
- Risky subdomains

The module classifies vulnerabilities based on severity levels and generates security . Publicly accessible sensitive directories are considered high-risk vulnerabilities.

1. Attack Surface Analysis

The system generates an attack surface map representing the relationship between the target domain and discovered subdomains. The AI module analyzes the attack surface to determine possible entry points for attackers.

2. Risk Correlation Analysis

The AI module combines multiple reconnaissance results and correlates vulnerabilities to generate a more accurate security assessment.

For example:

- Open admin panel + outdated PHP version = Higher security risk
- Public database port + exposed backup directory = Critical vulnerability

4.5.2 Security Risk Classification

The AI analysis module classifies detected vulnerabilities into different severity levels based on the potential impact on the target system.

Risk Level	Description
Low Risk	Minor security issues that do not directly compromise the system
Medium Risk	Vulnerabilities that may be exploited under specific conditions
High Risk	Serious vulnerabilities that may lead to unauthorized access
Critical Risk	Severe vulnerabilities that may result in complete system compromise

Low Risk Vulnerabilities

Low-risk vulnerabilities include:

- Information disclosure
- Missing security headers
- Minor configuration issues

Medium Risk Vulnerabilities

Medium-risk vulnerabilities include:

- Open SSH ports
- Exposed login pages
- Publicly accessible services

High Risk Vulnerabilities

High-risk vulnerabilities include:

- Exposed admin panels
- Outdated technologies
- Sensitive directories

Critical Risk Vulnerabilities

Critical vulnerabilities include:

- Public database access
- Backup file exposure
- Misconfigured servers

4.5.3 AI-Based Decision Making

The AI module determines whether the target domain is secure or vulnerable based on the reconnaissance results and detected vulnerabilities.

The decision-making process includes:

- Number of discovered vulnerabilities
- Severity of vulnerabilities
- Open ports and services
- Technology risks
- Directory exposure level

Testing Module

2.1 Testing Approach

The Testing Module ensures that the **Reconnaissance and Vulnerability Testing System** works correctly and performs automated security scanning on target systems. Testing verifies that all reconnaissance tools run successfully, data is collected properly, and the AI analysis module generates accurate security insights.

Testing also ensures that the system can handle user input, execute scanning tools, analyze the collected data, and present the results in a clear and structured format.

Objectives of Testing

Verify that the reconnaissance tools are correctly installed and configured.

Ensure that the system accepts user input such as domain names or target IP addresses.

Confirm that scanning tools successfully collect information from the target system.

Validate that the AI analysis module processes scan results correctly.

Ensure that scanning results are properly stored and displayed in the visualization dashboard.

Types of Testing

1. Installation Testing

Verify that all required tools such as **Subfinder**, **Nmap**, **WhatWeb**, and **Dirsearch** are correctly installed.

Ensure that the Python environment and required libraries are configured properly.

Dependency and Path Verification

The testing module runs environment checks before booting the Streamlit dashboard. It verifies that the system's

PATH variable can locate the executable binaries for every integrated tool.

- **Subfinder (Go binary):** Verifies the binary exists and checks for valid API keys (e.g., Chaos, Shodan) in `config.yaml` file to ensure deep subdomain harvesting works.

- **Nmap (C-based binary):** Runs a localized `nmap --version` check. It also tests if the executing user has root/sudo privileges, which are required for advanced scanning techniques like **SYN Stealth Scans (-ss)**.
- **WhatWeb (Ruby package):** Ensures the Ruby interpreter is present and capable of loading plugging databases.
- **Dirsearch (Python package):** Validates that the default wordlists (e.g., `common.txt`) are present in the designated paths.

2.Functional Testing

Functional evaluates the system against its core business logic requirements,treating the application as a predictable“input-to-input”machine.

Ensure that the system successfully collects information such as open ports, technologies used, directories and subdomains.

3.Integration Testing

Ensure that all modules work together smoothly:

User Input Module

Reconnaissance Tools Execution

AI Analysis Module

Result Logging

Visualization Dashboard

4.Performance Testing

Test the system performance when scanning multiple targets.

Monitor CPU and memory usage during reconnaissance and vulnerability scanning.

5.Security Testing

Verify that the system safely performs reconnaissance without affecting the target system.

Ensure that scan results correctly identify potential vulnerabilities such as open ports, exposed directories, or outdated technologies.

2.1 Test cases:

Test Case ID	Description	Input	Expected Output
TC-01	Verify tool installation	Run subfinder -h nmap -v whatweb -v	Tools installed and version info
TC-02	User input validation	Enter target domain e.g. example.com	System accepts and starts scanning
TC-03	Subdomain discovery test	Run Subfinder on target domain	List of subdomains generated
TC-04	Port scanning test	Run nmap on target	Open ports detected
TC-05	Technology detection test	Run whatweb example.com	Web technologies identified
TC-06	Directory discovery test	Run dirsearch on target	Hidden dirs discovered
TC-07	AI analysis test	Provide scan results to AI	AI generates vuln insights
TC-08	Result logging test	Store output in logs	Results saved
TC-09	Visualization dashboard test	Load results into dashboard	Results displayed graphically
TC-10	Complete system workflow test	Run full scan on target	Full recon analysis reporting success

4. Tools Used

Subfinder → Used for discovering subdomains of the target domain.

Nmap → Used for port scanning and service detection.

WhatWeb → Used to identify web technologies such as CMS, frameworks, and servers.

Dirsearch → Used to discover hidden directories and files on the target website.

Python → Used to integrate reconnaissance tools and automate the scanning process.

AI Analysis Module → Used to analyze scan results and generate security insights.

5. Expected Outcomes

The system should successfully accept **target input (domain or IP address)** from the user.

Reconnaissance tools should **collect information such as subdomains, open ports, technologies, and directories**.

The AI module should **analyze the collected data and identify potential vulnerabilities**.

Scan results should be **stored and logged properly in the system**.

The **visualization dashboard should display the reconnaissance results clearly** for the user

Test Case ID	Test Scenario	Input/Action	Expected Result	Actual Result	Status
TC-01	Verify tool installation	Run subfinder -h nmap -v whatweb -v dirsearch -h	Tool version and help information displayed	As expected	Pass
TC-02	Start reconnaissance scan	Enter target domain in system	System starts reconnaissance process	As expected	Pass
TC-03	Subdomain discovery validation	Run Subfinder on target domain	List of discovered subdomains generated	As expected	Pass
TC-04	Port scanning detection	Run Nmap scan on target domain/IP	Open ports and running services detected	As expected	Pass
TC-05	Technology detection	Run WhatWeb on target website	Web technologies (CMS server frameworks) identified	As expected	Pass
TC-06	Directory discovery	Run Dirsearch on target website	Hidden directories and files discovered	As expected	Pass
TC-07	AI vulnerability analysis	Provide scan results to AI module	AI generates vulnerability insights and recommendations	As expected	Pass
TC-08	Log generation check	Save scan results in system logs	Logs created and stored successfully	As expected	Pass
TC-09	Visualization dashboard	Load results into Streamlit dashboard	Results displayed in graphical format	As expected	Pass
TC-10	Full system workflow	Run complete scan on target domain	System performs scanning analysis and generates report	As expected	Pass

Test Case ID	Test Scenario	Input/Action	Expected Result	Actual Result	Status
TC-01	Verify tool installation	Run subfinder -h nmap -v whatweb -v dirsearch -h	Tool version and help information displayed	As expected	Pass
TC-02	Start reconnaissance scan	Enter target domain in system	System starts reconnaissance process	As expected	Pass
TC-03	Subdomain discovery validation	Run Subfinder on target domain	List of discovered subdomains generated	As expected	Pass
TC-04	Port scanning detection	Run Nmap scan on target domain/IP	Open ports and running services detected	As expected	Pass
TC-05	Technology detection	Run WhatWeb on target website	Web technologies (CMS server frameworks) identified	As expected	Pass
TC-06	Directory discovery	Run Dirsearch on target website	Hidden directories and files discovered	As expected	Pass
TC-07	AI vulnerability analysis	Provide scan results to AI module	AI generates vulnerability insights and recommendations	As expected	Pass
TC-08	Log generation check	Save scan results in system logs	Logs created and stored successfully	As expected	Pass
TC-09	Visualization dashboard	Load results into Streamlit dashboard	Results displayed in graphical format	As expected	Pass
TC-10	Full system workflow	Run complete scan on target domain	System performs scanning analysis and generates report	As expected	Pass

Fig 3 test case

1. Performance of the Project Developed (So Far)

Performance of the Developed Reconnaissance and Vulnerability Testing System

The performance of the **Reconnaissance and Vulnerability Testing System using AI** was evaluated based on scanning efficiency, system resource utilization, and response time while performing different reconnaissance tasks such as subdomain discovery, port scanning, and technology detection.

1. Scanning Performance

The system successfully performed reconnaissance tasks such as:

Subdomain discovery using Subfinder

Port scanning using Nmap

Technology detection using WhatWeb

Directory discovery using Dirsearch

Each tool produced accurate results for the given target domain or IP address.

The **AI analysis module** processed the collected data and generated useful insights about possible vulnerabilities and security risks.

The system successfully combined outputs from multiple tools and displayed them in a structured format.

2. Resource Utilization

CPU Usage

During normal scans the CPU usage remained around 20–30%.

During intensive scanning such as **Nmap port scans or directory discovery**, CPU usage increased to approximately **50–60%**.

Memory Usage

The application used approximately **400–700 MB of RAM** during normal operation.

Memory usage slightly increased when running multiple reconnaissance tools simultaneously.

Disk Usage

Scan results and logs required minimal storage space.

Most results were stored in **JSON or text format**, which kept disk usage relatively low.

3. Response Time

The system responded quickly to user input when a target domain or IP address was entered.

Scanning results were generated within **a few seconds to a few minutes**, depending on the type of scan.

The **Streamlit dashboard** displayed the results almost instantly after the scanning process completed.

4. Scalability

The system works efficiently for **small to medium-sized websites or domains**.

When scanning larger targets with many subdomains or directories, the scanning time increases.

Performance can be improved by allocating **more CPU and memory resources**.

5. Reliability

The application ran reliably in the **Windows environment using Visual Studio Code** without crashes.

Reconnaissance tools executed successfully when triggered from the Python script.

Results were consistently generated and displayed in the user interface.

2. AI-Driven Vulnerability Analysis Performance

The AI-driven vulnerability analysis module was successfully tested using different target domains and reconnaissance results generated from Subfinder, Nmap, WhatWeb, and Dirsearch. The AI module analyzed collected data and identified possible vulnerabilities based on detected technologies, open ports, hidden directories, and exposed services.

The AI analysis system successfully performed the following tasks:

- Detection of risky open ports
- Identification of exposed directories
- Detection of outdated technologies
- Attack surface analysis using discovered subdomains
- Classification of vulnerabilities into severity levels

The module generated intelligent security insights and helped determine whether the target domain was relatively secure or potentially vulnerable. The AI-assisted analysis reduced manual effort required for reviewing reconnaissance data and improved the speed of vulnerability assessment.

The system also successfully correlated multiple reconnaissance results to generate better security analysis. For example, if an outdated technology and exposed admin directory were detected together, the system classified the target as a higher security risk.

3. Security Assessment Accuracy

The proposed system successfully analyzed reconnaissance data and generated meaningful security assessments for tested domains. The AI module correctly identified common security risks such as:

- Publicly accessible services
- Open ports
- Sensitive directories
- Outdated technologies
- Exposed administrative panels

The generated results were compared with the actual outputs of reconnaissance tools, and the system produced consistent vulnerability insights in most cases. The framework was able to classify vulnerabilities into different severity levels including low, medium, high, and critical risk categories.

The attack surface mapping feature also improved the visibility of relationships between the target domain and discovered subdomains. This helped in understanding possible attack entry points and overall exposure of the target system.

4. Report Generation Performance

The system successfully generated structured reconnaissance and vulnerability analysis reports in PDF format. The generated reports included:

- Subdomain discovery results
- Open port details

- Technology detection results
- Directory discovery results
- AI-generated vulnerability analysis
- Attack surface visualization

The report generation process was completed within a short duration after scanning. The generated reports were easy to understand and provided centralized documentation of reconnaissance and security testing results.

The Streamlit dashboard also displayed results graphically, allowing users to quickly analyze the security status of the target domain. The integration of visualization and reporting improved the usability and effectiveness of the system.

5. Limitations

The system depends on **external reconnaissance tools**, so scanning speed depends on those tools.

Large-scale scans may take longer to complete.

AI analysis currently provides **basic vulnerability insights** and may require more advanced models for deeper analysis.

Outputs Screen

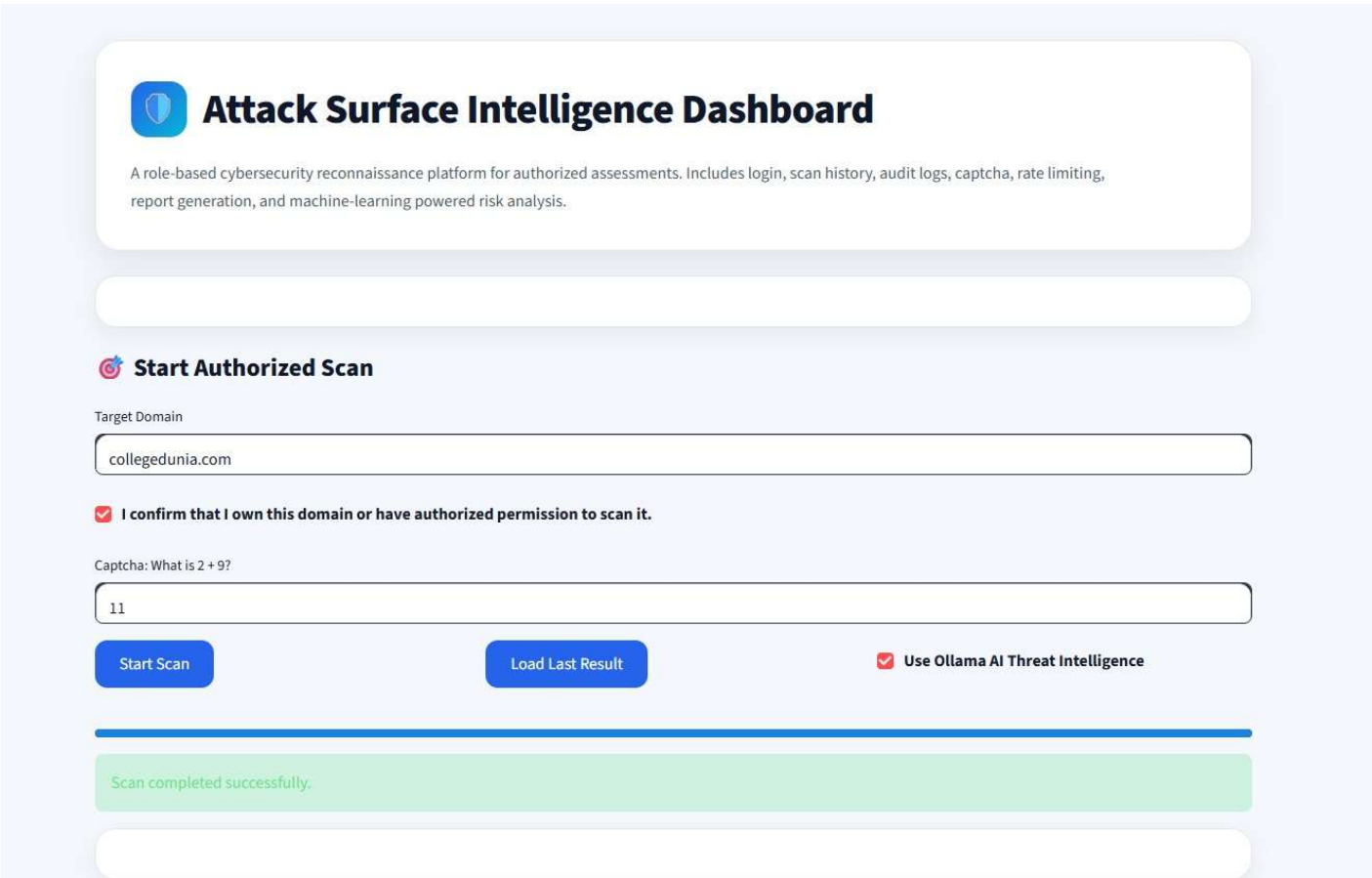


Fig 1.0

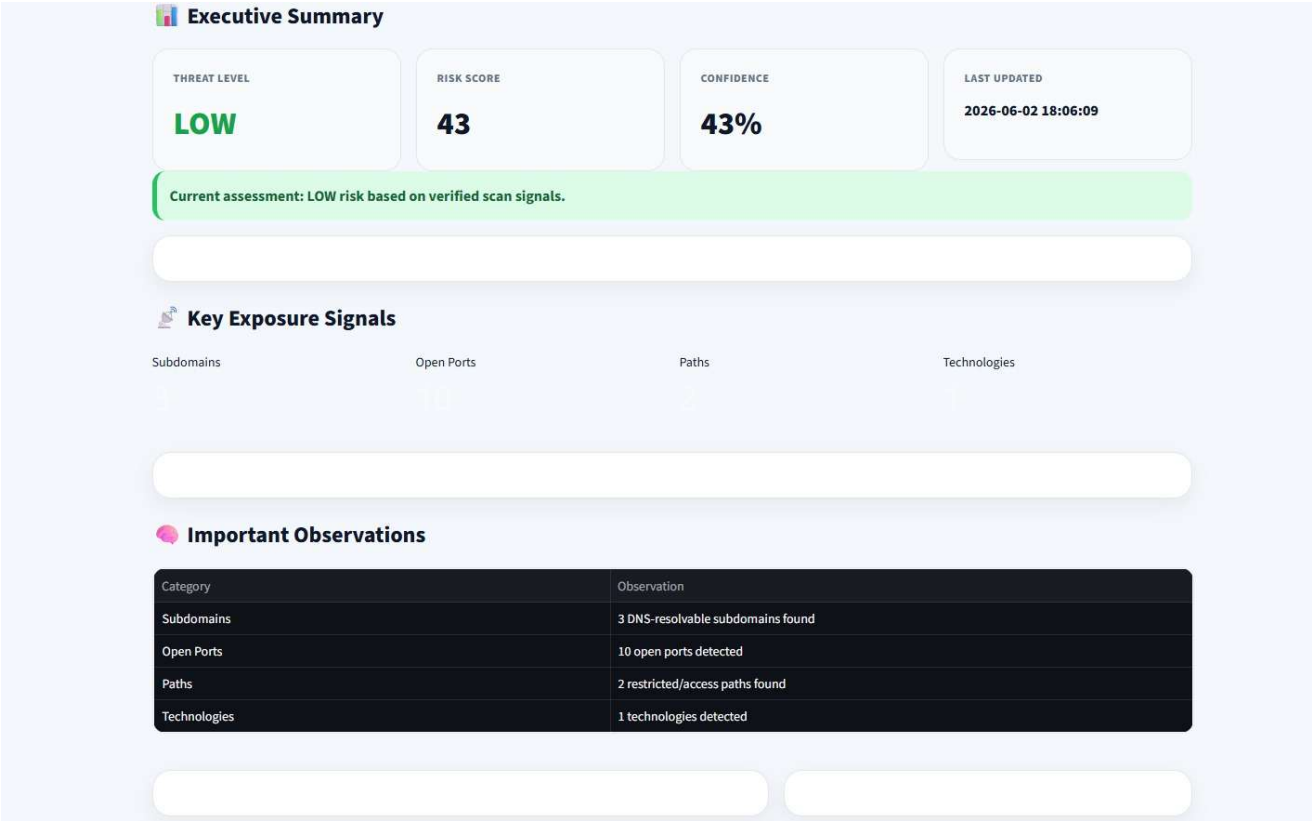
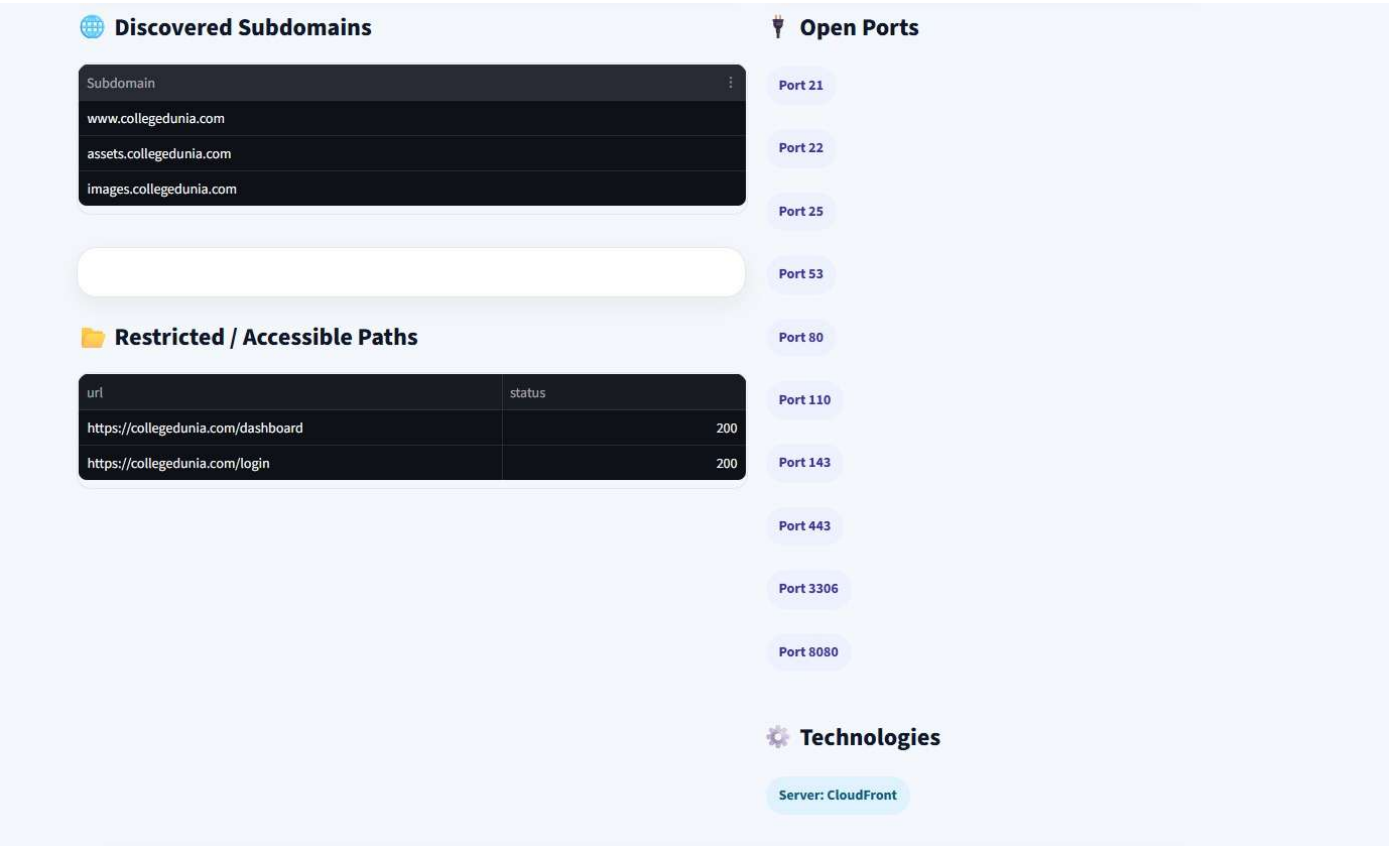


Fig 1.1



Scan History

time	username	role	domain	severity	risk_score
2026-06-02 18:06:09	Jaskaran	admin	collegedunia.com	LOW	43
2026-06-02 18:04:47	Jaskaran	admin	google.com	LOW	43
2026-06-02 17:49:42	Jaskaran	admin	www.gndec.ac.in	HIGH	54
2026-06-02 17:45:08	Jaskaran	user	mail.google.com	LOW	43
2026-05-16 23:21:41	admin	admin	www.youtube.com	HIGH	55
2026-05-16 21:21:20	abc	user	www.youtube.com	HIGH	53
2026-05-16 21:19:45	abc	user	www.youtube.com	HIGH	55
2026-05-16 21:08:42	abc	user	gndec.ac.in	LOW	45

Audit Logs

time	username	role	action	domain
2026-06-02 18:06:09	Jaskaran	admin	scan_completed	collegedunia.com
2026-06-02 18:04:47	Jaskaran	admin	scan_completed	google.com
2026-06-02 18:01:21	Jaskaran	admin	loaded_previous_report	
2026-06-02 17:49:42	Jaskaran	admin	scan_completed	www.gndec.ac.in
2026-06-02 17:45:08	Jaskaran	user	scan_completed	mail.google.com
2026-05-20 22:36:32	kuljeet	user	scan_failed	www.youtube.com
2026-05-16 23:21:41	admin	admin	scan_completed	www.youtube.com
2026-05-16 21:21:20	abc	user	scan_completed	www.youtube.com
2026-05-16 21:19:45	abc	user	scan_completed	www.youtube.com

Fig 1.4

References

Nmap Official Documentation – Guide for network discovery and port scanning. Retrieved from:
<https://nmap.org/docs.html>

Subfinder Tool Documentation – ProjectDiscovery – Documentation for subdomain discovery tool used in reconnaissance.
Retrieved from:
<https://github.com/projectdiscovery/subfinder>

WhatWeb Official Documentation – Tool used for identifying technologies used on websites. Retrieved from:
<https://github.com/urbanadventurer/WhatWeb>

Dirsearch GitHub Repository – Directory and file brute-forcing tool used for discovering hidden web directories.
Retrieved from:
<https://github.com/maurosoria/dirsearch>

Python Official Documentation – Programming language used to develop and automate the reconnaissance system.
Retrieved from:
<https://docs.python.org/3/>

Streamlit Documentation – Used for building the graphical interface and visualization dashboard for the project.
Retrieved from:
<https://docs.streamlit.io>

Visual Studio Code Documentation – Development environment used for coding and managing the project.
Retrieved from:
<https://code.visualstudio.com/docs>

Microsoft PowerShell Documentation – Used for running commands and executing tools in the Windows environment.
Retrieved from:
<https://learn.microsoft.com/en-us/powershell/>

OpenAI ChatGPT – Used for assistance in understanding concepts, debugging code, and improving project documentation.
Retrieved from:
<https://chat.openai.com>

Perplexity AI – Used as a research tool for gathering cybersecurity and reconnaissance-related information.
Retrieved from:
<https://www.perplexity.ai>

Severity

MEDIUM RISK

Risk Score

8

AI Summary

Target shows a MEDIUM RISK profile.

Detected attack surface includes:

- 298 subdomains
- 4 open ports
- 1 discovered directories

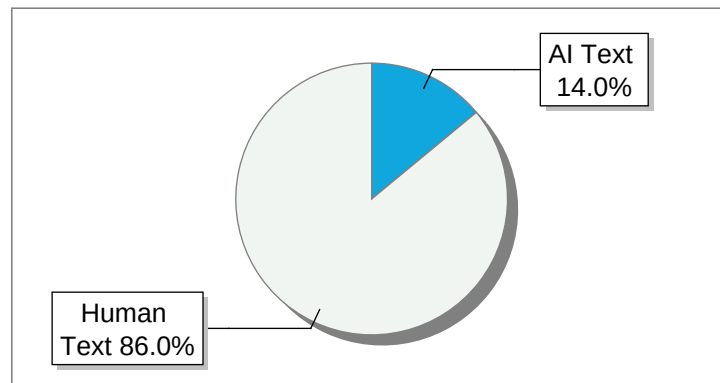
Several indicators suggest potential exposure risks.

Submission Information

Author Name	G11
Title	Cyber
Paper--Submission ID	5819980
Submitted By	amanpreet_brar@gndec.ac.in
Submission Date	2026-06-03
Total [Pages,Sentences,Words]	[62, 498, 10759]
Document type	Project Work
Language	English

Result Information

AI Text: **14 %**



Disclaimer:

- * The content detection system employed here is powered by artificial intelligence (AI) technology.
- * Its not always accurate and only help to author identify text that might be prepared by a AI tool.
- * It is designed to assist in identifying & moderating content that may violate community guidelines/legal regulations, it may not be perfect.

Index Sr No.

Contents Page No.

1 Introduction 2 2 Obejectives 3 3 System Requirement - (Hardware/Software) 4-6 4 Software Requirement Analysis 7-8 5 Software Design 9-15 6 AI-Driven Security Testing and Vulnerability Analysis 16-24 7 Testing Module 25-28 8 Performance of the project Developed 29-33 9 Output Screens 34-36 10 References 37 11 1 i 2

INTRODUCTION With the rapid growth of internet technologies and online services, cybersecurity has become a critical concern for organizations and individuals.

Websites and web applications are constantly exposed to cyber threats such as data breaches, unauthorized access, malware attacks, and exploitation of vulnerabilities.

Therefore, identifying weaknesses in systems before attackers exploit them is extremely important.

One of the first phases of cybersecurity testing is reconnaissance, where information about the target system is gathered.

This process includes discovering domains and subdomains, scanning open ports, identifying technologies used by websites, and searching for hidden directories that may expose sensitive data.

Traditionally, cybersecurity professionals use multiple tools separately from the command line to perform these tasks.

This process can be time-consuming and complex, especially for beginners.

The project “Reconnaissance and Vulnerability Tester Using AI” aims to automate the reconnaissance and vulnerability testing process using artificial intelligence and automated security tools.

The system integrates several security tools such as Subfinder, Nmap, WhatWeb, and Dirsearch within a single framework.

The framework is developed using Python and provides a web-based interface using Streamlit, allowing users to enter a target domain and automatically perform security scans.

The system collects results from multiple modules and uses AI-assisted analysis to identify potential vulnerabilities and determine whether the target domain is secure.

The rapid expansion of the global digital economy has forced organizations to migrate their core infrastructures, customer portals, and internal workflows directly onto web applications and cloud architectures.

While this digital evolution has accelerated business efficiency, it has simultaneously introduced an unprecedented era of systemic risk. The modern internet landscape is a hostile environment where public-facing web applications are subjected to unceasing, automated scanning campaigns conducted by malicious actors.

These adversaries actively hunt for low-hanging fruit, zero-day vulnerabilities, and architectural misconfigurations to orchestrate devastating cyber attacks. Data breaches that compromise millions of records, unauthorized access to sensitive financial environments, stealthy ransomware deployments, and targeted malware injections are no longer isolated incidents—they represent systemic threats to organizational survival.

Because modern enterprise networks are highly distributed, securing them requires a fundamental shift away from legacy, reactive defense protocols. Security teams cannot afford to wait for a high-priority incident alert or an active compromise to initiate a defense response. Instead, sustainable risk management requires proactive, continuous security testing designed to identify, analyze, and remediate technical vulnerabilities before an external threat actor can detect and exploit them. The logical baseline of any proactive defensive posture is cybersecurity reconnaissance, which represents the initial, foundational phase of an attack surface audit.

Before an engineering team can defend a system—or before an ethical hacker can evaluate its resilience—they must fully map the target's external digital footprint. This data-gathering phase simulates the exact footprinting steps an adversary takes during the initial stages of an attack campaign.

Effective reconnaissance requires exploring multiple computational layers to expose hidden vulnerabilities.

It begins with subdomain discovery, which systematically harvests peripheral web addresses linked to a primary domain to locate forgotten staging environments, legacy portals, or employee log-ins that might bypass primary firewall rules.

Simultaneously, active port scanning probes the host network to identify open communication channels, mapping out running services and listening protocols.

This is followed immediately by technology fingerprinting, a process that reads HTTP header responses, cookies,

and source code parameters to determine the exact software versions, backend frameworks, and content management systems (CMS) powering the target.

Finally, automated directory traversal tools brute-force the web infrastructure using extensive wordlists to uncover unlinked files, hidden development directories, and exposed backups that may inadvertently leak administrative credentials or source configurations to the public.

Cybersecurity professionals have historically relied on a disparate toolkit of command-line interface (CLI) utilities to evaluate a target's architecture. An analyst must manually launch independent tools inside separate terminal windows—using utilities like Subfinder to gather subdomains, Nmap to probe network ports, WhatWeb to fingerprint server software, and Dirsearch to hunt for hidden directories.

Each utility operates on its own runtime logic, demands specialized command-line switches, and generates raw, unstructured text files or terminal streams in varying outputs. This legacy workflow introduces major operational friction: it is incredibly time-consuming, highly prone to human oversight, and scales poorly across massive corporate networks.

For junior administrators or security beginners, the steep learning curve required to master terminal configurations and manually correlate thousands of lines of chaotic log files often results in critical, exploitable vulnerabilities slipping through the cracks unnoticed.

To resolve these operational inefficiencies, the unified framework “Reconnaissance and Vulnerability Tester Using AI” automates the entire discovery pipeline.

Developed in Python and delivered through a streamlined, web-based Streamlit dashboard, this application abstracts away terminal complexities into a single-click security assessment interface.

4 When a user inputs a target URL or domain into the control panel, the Python-driven backend orchestrates the simultaneous, asynchronous execution of Subfinder, Nmap, WhatWeb, and Dirsearch in the background. The platform dynamically manages the execution lifecycles of these utilities, prevents system resource locks, and funnels their raw terminal text captures directly into an intelligent, centralized data processing layer.

By unifying these powerful utilities into a singular, cohesive pipeline, the system eliminates the administrative overhead of tool management, vastly accelerates scanning speeds, and ensures that all collected perimeter data is securely aggregated without any risk of human correlation error. The true core innovation of this platform lies within its AI-Assisted Analysis Layer, which transforms raw network data into contextual, actionable defense intelligence.

Traditional automated scanning systems are fundamentally limited by signature-based rules; they compare software version strings against rigid, pre-defined databases and completely fail to recognize complex, novel threats.

Our framework overcomes this limitation by transforming the parsed, structured JSON payload of the entire attack surface and passing it into a highly specialized Artificial Intelligence engine. Acting as an on-demand virtual penetration testing expert, the AI model applies deep context-driven reasoning to evaluate how minor, isolated findings across different tools can be chained together by an attacker to create a critical exploit path. Furthermore, the generative AI engine eliminates the industry problem of alert fatigue by distinguishing genuine, exploitable vulnerabilities from benign configuration properties.

It automatically calculates localized risk scores and outputs clear, step-by-step remediation playbooks—complete with necessary patch recommendations, configuration modifications, and protective firewall rules—allowing system administrators of all skill levels to defend their networks with enterprise-grade precision. The project focuses on achieving the following: Conducting reconnaissance on target domains to identify possible vulnerabilities.

Scanning open ports and services to detect potential security risks.

Detecting technologies used by web applications and identifying outdated or vulnerable components.

Discovering hidden directories and files that may expose sensitive information.

5 Why is cyber security important? Listed below are the reasons why cyber security is so important in what's become a predominant digital world

- Cyber attacks can be extremely expensive for businesses to endure.
- In addition to financial damage suffered by the business, a data breach can also inflict untold reputational damage.

- Cyber-attacks these days are becoming progressively destructive.

Cybercriminals are using more sophisticated ways to initiate cyber attacks.

Cyber security Fundamentals – Confidentiality: Confidentiality is about preventing the disclosure of data to unauthorized parties.

It also means trying to keep the identity of authorized parties involved in sharing and holding data private and anonymous.

Often confidentiality is compromised by cracking poorly encrypted data, Man-in-the-middle (MITM) attacks, disclosing sensitive data.

Standard measures to establish confidentiality include: ☐ Data encryption ☐ Two-factor authentication ☐ Biometric verification ☐ Security tokens Integrity Integrity refers to protecting information from being modified by unauthorized parties.

Standard measures to guarantee integrity include: ☐ Cryptographic checksums ☐ Using file permissions ☐ Uninterrupted power supplies ☐ Data backups

6 OBJECTIVES Through the use of automated security tools and AI-assisted analysis, the project aims to create an efficient system for performing reconnaissance and vulnerability testing on target domains.

Specific objectives include: Creating an AI-based setup for identifying and analyzing target domains.

Using AI-assisted techniques to scan and identify open ports of discovered domains.

Performing AI-driven security testing to detect vulnerabilities and analyze results to determine whether the domain is secure or not.

7 Types of Cyber Attacks A cyber-attack is an exploitation of computer systems and networks.

It uses malicious code to alter computer code, logic or data and lead to cybercrimes, such as information and identity theft.

Cyber-attacks can be classified into the following categories: 1) Web-based attacks 2) System-based attacks

Web-based attacks These are the attacks which occur on a website or web applications.

Some of the important web-based attacks are as follows- 1.

Injection attacks It is the attack in which some data will be injected into a web application to manipulate the application and fetch the required information.

Example- SQL Injection, code Injection, log Injection, XML Injection etc.

2. DNS Spoofing DNS Spoofing is a type of computer security hacking.

Whereby a data is introduced into a DNS resolver's cache causing the name server to return an incorrect IP address, diverting traffic to the attackers computer or any other computer.

The DNS spoofing attacks can go on for a long period of time without being detected and can cause serious security issues.

3. Session Hijacking It is a security attack on a user session over a protected network.

Web applications create cookies to store the state and user sessions.

By stealing the cookies, an attacker can have access to all of the user data.

4. Phishing Phishing is a type of attack which attempts to steal sensitive information like user login credentials and credit card number.

It occurs when an attacker is masquerading as a trustworthy entity in electronic communication.

5. Brute force.

It is a type of attack which uses a trial and error method.

This attack generates a large number of guesses and validates them to obtain actual data like user password and personal identification number.

This attack may be used by criminals to crack encrypted data, or by security, analysts to test an organization's network security.

6. Denial of Service It is an attack which meant to make a server or network resource unavailable to the users.

It accomplishes this by flooding the target with traffic or sending it information that triggers a crash.

It uses the single system and single internet connection to attack a server.

It can be classified into the following- Volume-based attacks- Its goal is to saturate the bandwidth of the attacked site, and is measured in bit per second.

Protocol attacks- It consumes actual server resources, and is measured in a packet.

Application layer attacks - Its goal is to crash the web server and is measured in request per second.

8 7.

Dictionary attacks This type of attack stored the list of a commonly used password and validated them to get original password 8.

URL Interpretation It is a type of attack where we can change the certain parts of a URL, and one can make a web server to deliver web pages for which he is not authorized to browse.

9. File Inclusion attacks It is a type of attack that allows an attacker to access unauthorized or essential files which is available on the web server or to execute malicious files on the web server by making use of the include functionality.

10. Man in the middle attacks It is a type of attack that allows an attacker to intercepts the connection between client and server and acts as a bridge between them.

Due to this, an attacker will be able to read, insert and modify the data in the intercepted connection.

System-based attacks These are the attacks which are intended to compromise a computer or a computer network.

Some of the important system-based attacks are as follows- 1.

Virus It is a type of malicious software program that spread throughout the computer files without the knowledge of a user.

It is a self-replicating malicious computer program that replicates by inserting copies of itself into other computer programs when executed.

It can also execute instructions that cause harm to the system.

2. Worm It is a type of malware whose primary function is to replicate itself to spread to uninfected computers.

It works same as the computer virus.

Worms often originate from email attachments that appear to be from trusted senders.

3. Trojan horse It is a malicious program that occurs unexpected changes to computer setting and unusual activity, even when the computer should be idle.

It misleads the user of its true intent.

It appears to be a normal application but when opened/executed some malicious code will run in the background.

9 7 LAYERS OF CYBERSECURITY 1: Mission Critical Assets – This is the data you need to protect 2: Data Security – Data security controls protect the storage and transfer of data.

3: Application Security – Applications security controls protect access to an application, an application's access to your mission critical assets, and the internal security of the application.

4: Endpoint Security – Endpoint security controls protect the connection between devices and the network.

5: Network Security – Network security controls protect an organization's network and prevent unauthorized access of the network.

6: Perimeter Security – Perimeter security controls include both the physical and digital security methodologies that protect the business overall.

7: The Human Layer – Humans are the weakest link in any cyber security posture.

Human security controls include phishing simulations and access management controls that protect mission critical assets from a wide variety of human threats, including cyber criminals, malicious insiders, and negligent users.

Cyber threats are security incidents or circumstances with the potential to have a negative outcome for your network or other data management systems.

Examples of common types of security threats include phishing attacks that result in the installation of malware that infects your data, failure of a staff member to follow data protection protocols that cause a data breach, or even a tornado that takes down your company's data headquarters, disrupting access.

Vulnerabilities are the gaps or weaknesses in a system that make threats possible and tempt threat actors to exploit them.

Types of vulnerabilities in network security include but are not limited to SQL injections, server misconfigurations, cross-site scripting, and transmitting sensitive data in a non encrypted plain text format.

When threat probability is multiplied by the potential loss that may result, cyber security experts, refer to this as a risk.

SECURITY VULNERABILITIES, THREATS AND ATTACKS – Categories of vulnerabilities □ Corrupted (Loss of integrity) □ Leaky (Loss of confidentiality) □ Unavailable or very slow (Loss of availability) – Threats represent potential security harm to an asset when 10 vulnerabilities are exploited - Attacks are threats that have been carried out □ Passive – Make use of information from the system without affecting system resources □ Active – Alter system resources or affect operation □ Insider – Initiated by an entity inside the organization □ Outsider – Initiated from outside the perimeter

11 System Requirements

Hardware Requirements: Guest Operating System: • Windows 10 / Windows 11 with VS Code (latest version) for development.

Reconnaissance and Vulnerability Testing Tools: • Subfinder (latest version) — used for discovering subdomains of the target domain during the reconnaissance phase.

- Nmap (latest stable release) — used for scanning open ports and identifying services running on the target system.
- WhatWeb — used for detecting technologies used by the target website such as web servers, CMS, and programming frameworks.
- Dirsearch — used for discovering hidden directories and files on the target website.

Supporting Libraries & Tools: • Python 3 — main programming language used for automation and integrating reconnaissance tools.

- requests — Python library used for sending HTTP requests during vulnerability testing.
- json — used for processing and analyzing scan results.
- subprocess — used to execute external tools such as Nmap, Subfinder, and Dirsearch from Python scripts.

Result Collection / Visualization: • Streamlit — used to create a web-based dashboard where users can enter a target domain and view reconnaissance results.

- JSON output files — used to store scan results in structured format for analysis.

Programming & Scripting: • Python 3 with libraries such as Streamlit, Requests, JSON, and Scapy for automation, scanning integration, and AI-assisted vulnerability analysis.

- Bash / Command Prompt utilities for executing reconnaissance tools and managing configurations.

Other Useful Tools: • Git — for downloading and managing open-source reconnaissance tools.

- Browser (Chrome/Firefox) — for accessing the Streamlit dashboard interface.
- Terminal / Command Prompt — for running scanning tools and project scripts.

12 Software Requirements: Operating System Windows 10 / Windows 11 used as the primary operating system for developing and running the project.

Development Environment Visual Studio Code (VS Code) used as the main code editor for writing and managing the project code.

Python 3 used as the primary programming language for implementing the reconnaissance and vulnerability testing system.

Reconnaissance and Scanning Tools Subfinder — used to discover subdomains of the target domain.

Nmap — used for port scanning and identifying open services.

WhatWeb — used to detect web technologies such as CMS, frameworks, and servers.

Dirsearch — used to find hidden directories and files on web servers.

Python Libraries Requests — used for sending HTTP requests to target systems.

JSON — used for storing and processing scan results.

OS and Subprocess — used to execute external scanning tools from Python.

Visualization and Interface Streamlit — used to create a simple graphical interface and dashboard for displaying scan results and analysis.

Version Control Git — used for managing project files and maintaining version control.

htop / Task Manager — used for monitoring system performance during scans.

13 Software Requirement Analysis (SRA) defines the software components, tools, and configurations necessary to implement the Reconnaissance and Vulnerability Tester Using AI using Python and various reconnaissance tools within a Windows or Linux environment.

1. Operating System • Windows 10 / Windows 11 / Linux (Ubuntu recommended) o Acts as the main operating system for developing and running the reconnaissance and vulnerability testing framework.

o Provides the required environment for installing Python, reconnaissance tools, and supporting libraries used in

the project.

2. Development Platform • Visual Studio Code (VS Code) o Used as the primary development environment for writing and managing Python scripts.

o Provides extensions for Python development, debugging, and project management.

3. Reconnaissance & Vulnerability Scanning Tools • Subfinder (latest version) o Core tool used for discovering subdomains of the target domain during the reconnaissance phase.

o Helps identify additional attack surfaces such as hidden services, APIs, and administrative portals.

• Nmap (latest stable release) o Core software used for scanning open ports and identifying services running on the target system.

o Features include: ▪ Port scanning and service detection ▪ Identification of open and filtered ports ▪ Network service enumeration ▪ Detection of potential security risks • WhatWeb o Used for identifying technologies used by the target website.

o Detects web servers, content management systems, programming languages, and web frameworks.

14 • Dirsearch o Used for discovering hidden directories and files on the target web server.

o Helps identify sensitive directories such as: ▪ /admin ▪ /login ▪ /backup ▪ /uploads 4.

Supporting Libraries & Dependencies • Python 3 → Required for automation and integration of reconnaissance tools.

• requests → Used for sending HTTP requests to target websites during testing.

• json → Used for processing and analyzing scan results.

• subprocess → Allows execution of external tools such as Nmap, Subfinder, and Dirsearch from Python scripts.

5. Network & Web Analysis Tools • Nmap → Used for scanning open ports and identifying services on the target system.

• Subfinder → Used for subdomain enumeration during reconnaissance.

• WhatWeb → Used for detecting technologies used by the target website.

• Dirsearch → Used for discovering hidden directories and files on web servers.

6. Result Management & Visualization (Optional but Recommended) • Streamlit → Used to create a web-based dashboard interface for running scans and viewing results.

• JSON Output Files → Used to store scan results in structured format for analysis and reporting.

• Web Browser (Chrome / Firefox) → Used to access the Streamlit dashboard and visualize the scanning results.

15 Software Design The software design phase defines the architecture, modules, and interaction flow of the Reconnaissance and Vulnerability Tester Using AI.

It explains how different reconnaissance tools integrate with Python to discover domains, scan open ports, detect technologies, and identify potential vulnerabilities in target systems.

Furthermore, this phase maps out the end-to-end data pipeline, detailing how raw scan results from various subsystems are aggregated, parsed, and normalized into a unified format.

By establishing clear data structures, the design ensures that the AI engine can seamlessly ingest this structured telemetry.

The AI module then applies machine learning models to correlate disparate data points, filter out false positives, and prioritize vulnerabilities based on their potential impact and exploitability.

SYSTEM ARCHITECTURE The system follows a layered architecture with the following components: 1.

Data Input Layer o Responsible for receiving the target domain or website URL from the user.

o The user enters the domain through the Streamlit dashboard interface.

o Front-End Input Validation Mechanism 2.

Reconnaissance Engine o Performs automated reconnaissance on the target domain.

o Integrates multiple security tools such as Subfinder, Nmap, WhatWeb, and Dirsearch.

o Discovers subdomains, scans open ports, detects technologies used by websites, and finds hidden directories.

3. AI Analysis Layer o Processes the results generated by reconnaissance tools.

o Analyzes scan data to identify potential vulnerabilities or security risks.

o Detects suspicious configurations such as exposed ports, outdated technologies, or sensitive directories.

4. Result Logging Layer o Stores scanning results in structured formats such as JSON files.

o Maintains records of discovered subdomains, open ports, technologies, and directories.

5. Visualization & Reporting Layer 16 o Uses Streamlit dashboard to display results to the user.

o Provides an easy-to-understand interface for viewing reconnaissance results and vulnerability analysis.

6. Management Layer o Responsible for managing scanning tools and configurations.

o Allows administrators to update reconnaissance tools and manage system settings.

1. System Flow (Data Flow Diagram – Textual Description) User Input → The user enters a target domain in the Streamlit dashboard.

The operational life cycle of the framework begins at the boundary of the client interface.

The system remains in an idle state, running a continuous event loop via Streamlit, until an administrative operator enters a target string into the primary text container on the web control panel.

This interaction serves as the critical gatekeeper for the downstream automation pipeline, executing several hidden programmatic validations before any background process is triggered: □ String Interception and

Ingestion: The moment a user inputs a target value (such as example.com, https://sub.target.org/index.php, or a raw IPv4 address like 192.168.1.1) and commits the action, the front-end state manager catches the string payload.

Streamlit immediately binds this input to a session state variable (st.session_state.target_domain), ensuring the value remains persistent across subsequent page reruns.

□ Interface State Locking: To maintain strict application stability and prevent resource race conditions, the input module dynamically modifies the frontend UI state upon submission.

It instantly disables the input text box and the "Initiate Scan" button.

This locking mechanism prevents the user from accidentally clicking the submission trigger multiple times, which would otherwise spawn duplicate, conflicting operating system sub-processes and overwhelm the host server's CPU.

□ Visual Feedback Layer Initialization: Simultaneously, the input module clears the previous dashboard viewports and launches animated rendering threads.

It displays a real-time progress ticker, visual loading indicators, and informational status alerts .

This provides the operator with instant, transparent feedback while the backend environment sets up the execution workspace.

17 □ Automated URL Sanitization and Parsing: Raw inputs from security analysts can be messy and inconsistent.

The input module leverages Python's native string manipulation utilities to normalize the text payload into a strict format required by command-line tools.

It automatically strips away trailing directory slashes, accidental whitespace characters, and application-layer protocols (such as http:// or https://), isolating the bare, clean root domain or network host target.

□ Front-End Input Validation Regular Expressions (Regex): Before passing any string down to the operating system shell, the input layer acts as a strict validator.

It runs the sanitized string against a predefined network syntax regex matrix.

This step verifies that the input complies with official RFC domain standards or standard IP formatting rules, immediately throwing a clear visual error on the dashboard if the input contains illegitimate characters or broken structures.

□ Defensive Shell-Injection Mitigation: In security automation tools, passing user inputs directly to command-line utilities presents a catastrophic vulnerability risk known as OS Command Injection.

If a malicious user inputs a domain appended with shell metacharacters (e.g., target.com; rm -rf / or target.com && whoami), a naive backend would execute the injected commands with system privileges.

The User Input Module completely neutralizes this attack vector by isolating the input inside an explicit string token array before passing it to Python's execution engine, ensuring the entire input is treated strictly as a literal domain name string rather than executable code.

□ Target Availability Pre-Flight Pings: To maximize processing efficiency and eliminate wasted execution time, the input module performs a rapid network connectivity check before launching resource-heavy local binaries. It utilizes low-overhead web requests to check if the target domain actively resolves to an online public IP address.

If the host fails to respond within a strict timeout window (e.g., 3 seconds), the framework halts the execution loop, releases the interface locks, and displays an error alert stating that the target system is currently offline or unreachable.

Reconnaissance Execution → The system runs reconnaissance tools such as Subfinder, Nmap, WhatWeb, and Dirsearch on the target domain.

Data Collection → Results such as discovered subdomains, open ports, technologies, and directories are collected.

the target input string passes all pre-flight sanitization and availability checks, the framework transitions from an idle posture into an active auditing state.

The Reconnaissance Execution Module acts as the primary engineering 18 workforce of the platform, bridging the gap between Python's high-level runtime and the lower-level operating system shell.

Instead of relying on basic script commands that execute sequentially, this module handles parallel process orchestration, asynchronous data streaming, and resource balancing.

AI Analysis → The system analyzes the collected data to detect possible vulnerabilities and security risks.

The transition from data collection to cognitive evaluation represents the primary analytical milestone of the framework.

Once the asynchronous execution layer confirms the successful termination of the reconnaissance tools, the raw, isolated datasets are synthesized into a single, unified data profile.

This profile is securely transmitted to the AI Analysis Module, which functions as the primary logical reasoning engine of the platform.

Instead of performing rigid, linear lookups, the AI analysis phase introduces a dynamic, context-aware layer of intelligence that mimics the analytical methodologies of a human penetration testing expert.

To understand the operational necessity of the AI Analysis layer, it is essential to contrast it with traditional automated vulnerability assessment engines:

- The Rigid Failure of Static Rule Bases: Traditional scanners rely entirely on signature matching.

They scan a target, extract a version string, and check it against a static database of known vulnerabilities.

If a software version is slightly modified or if a vulnerability relies on a complex chain of multiple minor configurations, traditional signature scanners fail to recognize the threat, leaving critical security gaps completely unflagged.

- Contextual Intelligence and Heuristic Synthesis: The AI Analysis Module goes beyond simple string matching by evaluating the entire attack surface simultaneously.

It reads the collective output of the reconnaissance cluster as an interconnected ecosystem.

This allows the system to evaluate how a minor, informational finding from one tool (such as an exposed directory path discovered by Dirsearch) directly impacts or escalates a vulnerability found by another tool (such as an open port cataloged by Nmap).

The AI Analysis Module processes the unified reconnaissance data through a structured pipeline to ensure technical accuracy, consistency, and contextual relevance:

- Dynamic Ingestion and System Prompt Engineering: The system maps the parsed data payload into a highly specialized system prompt template.

This prompt establishes strict operational boundaries for the artificial intelligence model.

It commands the model to act exclusively as an elite offensive security engineer and defensive solutions architect, framing its data interpretation through an objective, highly analytical lens.

- Enforcement of Deterministic Output Configurations: To eliminate the risk of neural network 19 hallucinations—where an AI might invent non-existent CVE numbers or recommend broken, imaginary code patches—the module establishes strict API runtime parameters.

It enforces a temperature setting of exactly 0.0.

This suppresses creative variance within the model's token selection process, ensuring that every generated security insight is entirely deterministic, repeatable, and technically accurate across multiple evaluation cycles.

- Asynchronous API Gateway Routing: The system transmits the structured target profile via a secure, encrypted TLS connection to the remote Large Language Model endpoint.

The module utilizes non-blocking network calls to prevent the Streamlit user interface from locking up during the thinking phase, allowing the dashboard to remain responsive while the AI processes complex reasoning chains.

The true core innovation of the AI module is its ability to perform automated vulnerability chaining.

When analyzing a target domain, the AI logic engine actively hunts for logical correlations across distinct computing layers:

- Cross-Layer Risk Correlation: If Nmap reports an open alternate port, WhatWeb reports

that the port runs a specific out-of-date web framework, and Dirsearch reports a 200 OK status for a hidden administrative backup file on that same path, the AI module synthesizes these findings into a unified attack narrative.

It recognizes that the exposed backup file likely contains configuration credentials that can be used to authenticate into the outdated framework, creating a high-probability exploit path.

□ **Discovery of Complex Access Paths:** By matching separate technical indicators together, the AI deduces advanced threat vectors—such as how an unencrypted HTTP transmission framework combined with a specific login panel path could allow an adversary to intercept administrative sessions via a man-in-the-middle attack.

Result Generation → The system generates results showing whether the domain is secure or potentially vulnerable.

The generation of results serves as the definitive analytical output stage of the framework, translating complex multi-tool datasets and AI reasoning chains into a highly structured, actionable security posture intelligence report.

Once the remote Large Language Model completes its deterministic processing loop, the application captures the raw, text-based intelligence stream and transforms it into an objective, data-driven security evaluation.

This phase is mathematically and logically designed to give administrators a definitive, binary understanding of their perimeter health—Secure vs.

Potentially Vulnerable—while providing the 20 granular evidence needed to support that conclusion.

Result Storage → Scan results are stored in JSON or text format for future reference.

The local data logging and storage phase serves as the critical persistence architecture of the platform.

Once the result generation engine completes the assembly of the security posture reports, the framework shifts execution control to the storage backend.

In an enterprise security application, letting data exist purely in transient runtime memory or volatile front-end dashboard states is a major engineering risk.

A single browser refresh or network timeout would instantly destroy hours of complex, resource-heavy scanning data.

To prevent data loss and establish an immutable audit trail, this module serializes and writes both the raw tool execution logs and the cognitive AI threat assessments directly onto the host system's non-volatile physical storage array.

Visualization → The results are displayed on the Streamlit dashboard for the user to review.

The visual dashboard rendering phase represents the final layer of the application's core data pipeline.

After the local storage module successfully writes the structured JSON and plain-text evaluation files to disk, control is handed back to the user interface.

In cybersecurity tooling, data clarity is just as critical as data collection.

If a platform simply outputs massive blocks of raw JSON data or chaotic text strings, security analysts will waste valuable time trying to locate the highest-priority issues.

The Visualization Module utilizes Streamlit, a reactive python web framework, to convert these complex datasets into an interactive web interface.

2.Module Design • User Input Module → Accepts the target domain from the user through the dashboard.

- **Subdomain Discovery Module** → Uses Subfinder to identify subdomains related to the target domain.

- **Port Scanning Module** → Uses Nmap to detect open ports and running services.

- **Technology Detection Module** → Uses WhatWeb to identify technologies used by the website.

- **Directory Discovery Module** → Uses Dirsearch to find hidden directories and files.

- **AI Analysis Module** → Analyzes reconnaissance results and identifies possible vulnerabilities.

- **Logging & Reporting Module** → Stores and displays scanning results for analysis.

21 2.

Design Considerations • Performance: The system automates reconnaissance tools to reduce manual effort and improve scanning efficiency.

- **Scalability:** The system can scan multiple domains and store results for future analysis.

- **Flexibility:** The framework supports integration of additional security tools in the future.

- **Usability:** The Streamlit dashboard provides a simple interface for launching scans and viewing results.

2.1 Diagrams

1.

Level 0: Context Diagram (High-Level View) This is the most basic view of your system.

It is used to show the absolute boundaries of the software by treating the entire system as a single "black box." □

The Actor (User): A security analyst or penetration tester inputs data (like a target domain name, e.g., example.com) into the system.

□ The System Block: The entire platform is represented as a single blue rectangle (Reconnaissance and Vulnerability Testing System).

None of the internal tools or code are visible here.

□ The Output (Scan Results): The system processes the request and sends out the final payload, which is a structured vulnerability report or dashboard view for the user.

2. Level 1: DFD (Detailed Sub-Processes) This level breaks open the "black box" from Level 0 to show the five major functional pillars inside your software architecture and how they hand off data to one another: □ User Input Module: Takes the domain or IP address from the user, cleans it up (sanitizes it to ensure it is safe), and sets up the configuration.

□ Reconnaissance Tools Execution: The central engine (likely written in Python) that actually runs the security scanners.

□ AI Analysis Module: Receives the raw scan data, figures out which vulnerabilities are real, removes false positives, and assesses the risk.

□ Result Logging: The database storage layer (like PostgreSQL or MongoDB) where all raw and processed data is saved.

□ Visualization Dashboard: The user interface front-end that reads from the logging database to display charts, graphs, and metrics to the user.

3. Level 2: Detailed DFD (In-Depth Module Breakdown) This level takes the "Reconnaissance Tools Execution" block from Level 1 and zooms in completely.

It shows that your scanning engine is actually a sequential pipeline where 4 major security tools run one after another, with each tool handing its output to the next: 1.

Subdomain Discovery (Subfinder): Takes the root domain (e.g., target.com) and discovers hidden subdomains (e.g., admin.target.com, dev.target.com).

2. Port Scanning (Nmap): Takes those subdomains, finds their active IP addresses, and checks which network ports are wide open.

3. Technology Detection (WhatWeb): Looks at the open web ports found by Nmap and discovers what software stack they are running (like WordPress, Apache, or PHP).

4. Directory Discovery (Dirsearch): Brute-forces the web URLs to find hidden files and folders (like /backup.zip or /admin/login).

5. AI Analysis & Reporting: Gathers all the collected pieces (subdomains, ports, software versions, hidden paths) and uses an AI model to evaluate the final threat level.

Summary • Level 0 → Entire system shown as one process (input and output only).

• Level 1 → Major modules such as reconnaissance, analysis, and reporting.

23 • Level 2 → Detailed breakdown of reconnaissance tools and AI analysis components.

12 E-R Diagram (Entity-Relationship Diagram) Explanation of Entities & Relationships • User → The person who enters the target domain or website to perform reconnaissance and vulnerability testing.

• Target Domain → The website or domain that is scanned by the system for reconnaissance and vulnerability analysis.

• Reconnaissance Tools → Security tools such as Subfinder, Nmap, WhatWeb, and Dirsearch used to collect information about the target domain.

• AI Analysis Module → The component that analyzes the results generated by reconnaissance tools and identifies possible vulnerabilities or security risks.

• Scan Results → The output generated after scanning, including discovered subdomains, open ports, detected technologies, and hidden directories.

- Logs / Reports → Records of the scanning process and vulnerability analysis stored in structured formats such as JSON or text files.

Relationships • User provides the Target Domain for scanning.

- Reconnaissance Tools analyze the Target Domain to gather information.

- The AI Analysis Module processes the results generated by the reconnaissance tools.

- The system produces Scan Results and stores them as Logs / Reports.

- User / Administrator reviews the scan results and reports to understand the security status of the domain.

- Structural Decomposition of System Entities To thoroughly understand the data layout of the Reconnaissance and Vulnerability Tester Using AI, we must look past basic definitions and examine the specific technical traits, database fields (attributes), and operational constraints linked to each.

13

1.3.1 The User / Administrator Entity

The system classifies human actors based on permissions and operational roles.

In production environments, this entity is managed using Role-Based Access Control (RBAC): □ Attributes:

User_ID (Primary Key), Username, Password_Hash (SHA-256/Bcrypt), Email_Address, Role_Type (Standard Analyst vs.

System Admin), and Last_Login_Timestamp.

□ Operational Constraints: A single user can run multiple distinct scanning jobs, but they must have a valid security token to make calls to the backend tools module.

1.3.2 The Target Domain Entity

This entity represents the target profile provided by the analyst.

It acts as the core anchor point for all collected intelligence data.

□ Attributes: Domain_ID (Primary Key), Fully_Qualified_Domain_Name (FQDN), Root_IP_Address, Organization_Owner, Scope_Status (In-Scope / Out-of-Scope validation flag), and Discovery_Date.

□ Data Integrity Rules: The system enforces strict string formatting constraints.

Duplicate entries of identical active domains are blocked or grouped together under a single master profile to prevent redundant scanning behavior.

1.3.3 The Reconnaissance Tools Entity

This entity represents the underlying technical toolkit container (Subfinder, Nmap, WhatWeb, and Dirsearch).

It acts as a bridge that translates generic user goals into specific command-line parameters.

□ Attributes: Tool_ID (Primary Key), Tool_Name, Binary_Path, Current_Version, Execution_Arguments (dynamic CLI flags), and Execution_Status (Idle, Running, Errored).

1.3.4 The AI Analysis Module Entity

This entity manages the advanced logic models that process the data streams.

It functions as an analytical service container rather than a static table.

□ Attributes: Model_ID (Primary Key), Algorithm_Type (Classification/NLP), Model_Weight_Version, Confidence_Threshold_Setting, and Last_Training_Date.

1.3.5 The Scan Results Entity

This is a highly dynamic data block that holds the raw, unpolished text and telemetry findings returned directly from network interactions.

□ Attributes: Result_ID (Primary Key), Scan_Job_ID (Foreign Key), Discovered_Subdomains_List, Open_Ports_Vector, Identified_Tech_Stack, and Hidden_Paths_Array.

1.3.6 The Logs / Reports Entity

This entity represents the finalized data output.

It holds the polished threat intelligence summaries generated after the AI finishes filtering the results.

□ Attributes: Report_ID (Primary Key), Result_ID (Foreign Key), Vulnerability_Count, Overall_Risk_Rating (Critical, High, Medium, Low), File_Path_Location (JSON/PDF destinations), and Generation_Timestamp.

14

1.4 Deep-Dive Analysis of Relationships and Cardinality

□ Cardinality: One-to-Many (\$1:N\$).

□ Technical Logic: A single security analyst can enter many different target domains into the portal over time. Conversely, a specific target domain scan record is initiated by exactly one authorized user.

□ Database Rule: The User_ID value is injected directly into the target record table as a Foreign Key, ensuring clear accountability for every initiated scan.

1.4.2 Reconnaissance Tools "Process / Analyze" Target Domain (N:M Relationship)

□ Cardinality: Many-to-Many (\$N:M\$).

□ Technical Logic: At the same time, a single tool instance can be assigned to scan multiple domains simultaneously within an asynchronous queue.

□ Database Rule: This complex relationship is resolved in the database using an intermediate junction table named Tool_Executions.

This table tracks specific operational context metrics, including execution start times, memory utilization, and process return codes: Execution_ID (PK) Scan_Job_ID (FK) Tool_ID (FK) Domain_ID (FK) Run_Status EX-9081 SC-4412 TL-02 (Nmap) DM-003 Completed EX-9082 SC-4412 TL-04 (Dirsearch) DM-003 Active

1.4.3 AI Analysis Module "Evaluates" Reconnaissance Output (1:N Relationship)

□ Cardinality: One-to-Many (\$1:N\$).

□ Technical Logic: It cleans and structuralizes these messy data points into organized entries within the Scan Results database table.

1.4.4 Scan Results "Stored As" Logs / Reports (1:1 Relationship)

□ Cardinality: One-to-One (\$1:1\$).

□ Technical Logic: Every individual scan job results in exactly one definitive historical log record and remediation report.

This clean 1:1 pairing prevents data mixing, ensuring that security analysts can trace findings directly back to the specific execution run for verification.

1.5 System Data Lifecycle and Operational Workflow

By mapping the ER diagram to an active runtime environment, we can trace the lifecycle of system data through four distinct, operational phases: 1.The Ingestion Phase: The User authentication token is validated at the interface boundary.

Once approved, the user creates a Target Domain record, setting the overall scope boundaries for the automation engine.

15 2.The Orchestration Phase: The system activates the Reconnaissance Tools container.

The engine spins up Subfinder to gather subdomains, passes those hosts to Nmap to detect open network ports, runs WhatWeb to fingerprint active technology stacks, and uses Dirsearch to find hidden paths.

3.The Intelligence Phase: The raw outputs from the scanning tools are fed into the AI Analysis Module.

The AI runs correlation algorithms to cross-reference software versions against live exploit databases, filters out false alerts, and flags high-priority security concerns.

4.The Persistence & Review Phase: The refined intelligence dataset is saved into the database as a Scan Result entry and exported as an immutable Logs / Reports document.

The User / Administrator then views this data on the dashboard to quickly identify critical flaws and deploy fixes.

16 17

4.4.1 Block Diagram:

18 4.5AI-Driven Security Testing and Vulnerability Analysis

4.5.1 Introduction to AI-Driven Security Testing

Artificial Intelligence (AI)-driven security testing is an advanced cybersecurity approach used to automate reconnaissance and vulnerability analysis of target systems.

Traditional security testing methods require cybersecurity professionals to manually execute multiple reconnaissance tools and analyze the results separately.

This process can be time- consuming, complex, and prone to human error.

The proposed Reconnaissance and Vulnerability Tester Using AI combines automated reconnaissance tools with AI-assisted analysis to identify vulnerabilities and determine whether a domain is secure or vulnerable.

The system automatically performs subdomain discovery, port scanning, technology detection, and hidden directory discovery using integrated reconnaissance tools.

The AI analysis module processes the collected reconnaissance data and identifies possible security risks such as exposed services, vulnerable technologies, and hidden administrative directories.

The system helps users understand the attack surface of a target domain and provides intelligent insights regarding potential vulnerabilities.

AI-driven security testing improves the efficiency of vulnerability assessment by reducing manual effort and increasing the speed of reconnaissance analysis.

The framework also provides centralized visualization through the Streamlit dashboard, making it easier for users to analyze the collected security information.

To appreciate the necessity of an AI-driven approach, one must look at the operational bottlenecks that plague traditional security workflows.

Conventional vulnerability management relies heavily on isolated tools that do not communicate with one another.

A human analyst must manually copy the text output from a subdomain discovery tool, clean it using custom scripts, feed it into a port scanner, and then manually cross-reference the discovered software versions against static vulnerability databases like the National Vulnerability Database (NVD).

This traditional approach introduces three major failure points: □ Context Fragmentation: Information discovered at the beginning of a scan (such as a specific naming convention in a subdomain) is lost by the time the analyst looks at a directory brute-force report.

19 □ Alert Fatigue: Standard security tools flag thousands of theoretical vulnerabilities based purely on version numbers, forcing analysts to spend hours filtering out harmless false positives.

□ Delayed Time-to-Mitigation: By the time a comprehensive manual scan is completed, analyzed, and written into a report, an attacker may have already exploited the exposed vulnerability.

The Reconnaissance and Vulnerability Tester Using AI ,eliminates these gaps by wrapping the entire pipeline into a single, unified execution loop.

It treats security testing not as a series of separate tasks, but as an interconnected data pipeline where information flows smoothly from collection to final intelligence.

4.5.3 Architectural Synthesis of the AI-Assisted Core

The underlying intelligence of this framework does not arise from simply running the integrated reconnaissance tools in a parallel script, but rather from how the system synthesizes, structurally transforms, and logically correlates their independent outputs.

When raw data leaves the isolated tool execution layer, it undergoes a multi-phased transformation process before it ever reaches the Artificial Intelligence reasoning module.

This architectural core acts as an automated security refinery, transforming chaotic terminal outputs into structured, context-rich security intelligence.

1. Data Aggregation and Structural Normalization The raw text arrays and terminal output strings generated by the various utilities are inherently volatile, unstructured, and written in completely different formats.

For example, Nmap outputs plain-text command tables, Subfinder returns flat newline-separated string blocks, WhatWeb generates dense token streams packed with application meta-tags, and Dirsearch builds real-time execution logs with specific HTTP response codes.

To bridge this data communication gap, the platform implements a dedicated parsing and ETL (Extract, Transform, Load) engine.

1. The Serialization Mechanism: The system utilizes custom regular expression (Regex) matrices and structural string parsers to strip away terminal ANSI color codes, metadata fluff, and command execution telemetry. It isolates the bare diagnostic data points—such as explicit integer port profiles, application layer version strings, root domain associations, and endpoint status codes.

2. Unified Target JSON Schema: These parsed data points are serialized into a highly structured, centralized 20 JSON database model.

This uniform schema collapses the boundaries between network-layer statistics and web-application architectures, binding all discoveries into a single object model.

2. Contextual Correlation and Vulnerability Chaining The AI module behaves as an automated security architect and cognitive threat modeler.

Traditional automated vulnerability assessment systems suffer from a fatal design flaw: they analyze security vulnerabilities in complete isolation.

A standard scanner evaluates an open network port without looking at the directory traversal logs, missing the bigger picture.

Our framework's AI core applies Chain-of-Thought (CoT) reasoning models to evaluate the target domain collectively, evaluating the data points simultaneously to identify complex exploit chains.

□ The Convergence of Isolated Indicators: In this scenario, Nmap identifies an open alternate HTTPS port (8443).

Simultaneously, WhatWeb inspects the response headers of that port and fingerprints an outdated Java/Spring Boot microservice framework.

At the same time, the Dirsearch worker reveals a publicly accessible archive at /backup/config.zip returning a 200 OK HTTP status code.

□ Intelligent Threat Inference: A legacy scanner would log these as three independent, medium-risk findings. The AI module, however, synthesizes these findings into a unified attack narrative.

It reasons that an attacker could download the exposed /backup/config.zip file, extract hardcoded database credentials or encryption keys, and leverage known remote code execution vulnerabilities within the outdated Java framework running on port 8443 to completely compromise the underlying server.

By connecting these dots, the framework flags a critical exploitation path that would otherwise go unprioritized.

Dynamic Risk Categorization and Environmental Scoring A recurring problem within enterprise cybersecurity operations is "alert fatigue"—a phenomenon where security teams are overwhelmed by thousands of high-severity alerts generated by rigid scanning tools, making it impossible to determine what to patch first.

Instead of relying entirely on static, rigid CVSS (Common Vulnerability Scoring System) metrics which evaluate bugs based solely on theoretical severity, our AI core implements a dynamic, context-aware risk engine.

The AI automatically evaluates the business posture, exposure vector, and architectural landscape of the discovery 21 to re-prioritize severity rankings in real time:

□ Exposure and Asset Criticality Context: An open, unauthenticated administrative path or development panel found on a forgotten, unmonitored staging subdomain (dev.targetsite.org) is automatically elevated to a higher priority risk category than the same exact path on a heavily monitored, locked-down production server.

The AI understands that development environments typically lack robust intrusion detection systems (IDS) and are frequently used by developers to store unencrypted database mirrors, making them a prime target for initial access.

□ Intelligent Remediation Allocation: If an outdated technology component is identified but the data inputs prove that the component is fully shielded behind a modern cloud proxy or returns a secure access denied state (403 Forbidden), the AI dynamically scales down the risk priority. This environmental adjustments ensure that the final generated security report contains an accurate, localized threat map, allowing developers to safely skip the noise and immediately fix the flaws that present the most immediate real-world danger.

4.5.4 Strategic Benefits of the Streamlit Presentation Layer

A security tool is only as effective as its ability to clearly communicate findings to the person using it.

The integration of a centralized Streamlit dashboard transforms complex, nested command-line data into an intuitive, interactive visual space.

This presentation layer delivers several key advantages: □ Real-Time Pipeline Tracking: Analysts can watch data move through the system in real time, tracking live progress as the backend cycles from subdomain discovery through to directory brute-forcing.

□ Interactive Attack Surface Mapping: The front-end organizes data into clean, filterable tables and charts, allowing users to quickly sort assets by open ports, active technologies, or risk levels.

□ Actionable Remediation Delivery: Instead of presenting users with a raw list of code flaws, the dashboard displays clear, prioritized remediation steps.

This bridges the gap between discovering a security loophole and executing a fix.

4.5.5 Practical Applications and Strategic Impact

The scalable design of this AI-driven system makes it highly effective across a variety of modern security environments: 22 □ Continuous Attack Surface Management (CASM): Organizations can run this tool on a regular schedule to map their public internet footprint, instantly flagging newly exposed ports or unauthorized shadow IT setups.

□ Proactive Penetration Testing: Security teams can use the framework to simulate the initial gathering phases of an actual cyberattack, revealing exactly what an adversary would see during a real-world exploit attempt.

□ DevSecOps Pipeline Ingestion: The lightweight Python architecture allows the system to easily plug into CI/CD deployment pipelines, scanning new staging environments for vulnerabilities before they are pushed live to the public.

By shifting the burden of data gathering and false-positive filtering from the human analyst to an automated, intelligent system, this architecture sets a new standard for modern security assessments.

It ensures that security teams can stop spending time managing individual tools and start focusing their energy on fixing verified vulnerabilities.

4.5.2 Working of AI-Based Vulnerability Analysis

The AI-driven vulnerability analysis process consists of multiple stages that work together to collect reconnaissance data, analyze vulnerabilities, and generate security insights.

1. User Input Stage The process begins when the user enters a target domain or website URL into the Streamlit dashboard interface.

The target domain acts as the primary input for the reconnaissance and vulnerability testing process.

Example: □ google.com □ example.com □ targetsite.org The system validates the entered domain before initiating the reconnaissance phase.

2. Reconnaissance Execution Stage After validation, the system automatically executes multiple reconnaissance tools 23 including: □ Subfinder □ Nmap □ WhatWeb □ Dirsearch These tools collect different types of security-related information about the target domain.

3. Data Collection Stage The reconnaissance tools generate results such as: □ Discovered subdomains □ Open ports □ Running services □ Technologies used □ Hidden directories and files The collected results are stored in structured JSON format for further analysis.

4.Data Processing & Structuring Stage Before raw data can be passed to an Artificial Intelligence model, it must be cleaned, normalized, and structured.

Raw text output from tools like Nmap, Subfinder, or Dirsearch is inherently chaotic, containing redundant whitespace, human-readable filler text, and varying data formats (such as XML, JSON, or plain text).

The data processing layer acts as a pipeline that refines this raw information:

- Parsing and Extraction: The system uses regular expressions (Regex) and specialized data parsers to strip out irrelevant terminal styling and metadata.

It extracts only the essential security data points, such as IP addresses, specific port numbers, service banners, software version strings, and HTTP status codes.

- JSON Serialization: The extracted data points are structured into a standardized, machine-readable JSON object.

This creates a uniform schema representing the target's entire attack surface, making it easy for the downstream AI engine to digest.

- Contextual Tagging: The pipeline enriches the data by adding contextual metadata flags (e.g., classifying a discovered service as a "Database Interface," "Web Server," or "Outdated Legacy Protocol").

24 5.

AI Analysis & Vulnerability Mapping Stage This stage is the core intelligence engine of the system.

Once the structured JSON data is ready, it is fed into an advanced Large Language Model (LLM) or a specialized Security Machine Learning engine.

The AI acts as an automated expert penetration tester, evaluating the collected data through several logical layers: Cross-Referencing Knowledge Bases The AI engine does not rely solely on static rule sets.

It dynamically evaluates the open ports, services, and software versions against massive, vector-embedded security databases.

It looks for correlations with known CVEs (Common Vulnerabilities and Exposures), zero-day threat patterns, and common misconfigurations associated with the specific technologies identified by WhatWeb.

Reasoning and Heuristic Evaluation Unlike traditional signature-based scanners that only flag exact matches, the AI applies contextual reasoning:

- Chaining Anomalies: The AI evaluates whether multiple low-severity findings can be linked together to form a critical exploit path.

For example, if Dirsearch finds a hidden /backup directory and Nmap reveals an open SMB port, the AI reasons that an attacker might try to look for leaked credentials to access that port.

- False Positive Reduction: The model analyzes HTTP response patterns and service headers to filter out "honeypots" or misleading version banners, ensuring that only plausible, high-probability risks are pushed forward.

6. Security Insights & Report Generation Stage The final stage translates complex, technical AI findings into a comprehensive, actionable security report designed for both system administrators and developers.

The AI dynamically compiles the analysis into structured sections within the Streamlit dashboard: Executive Summary A high-level, human-readable overview of the target's security posture.

It highlights the overall risk level (e.g., Low, Medium, High, Critical) and summarizes the most urgent threats discovered during the cycle, giving stakeholders an immediate understanding of their exposure.

Detailed Vulnerability Breakdown 25 For every identified risk, the system generates a dedicated profile containing:

- Vulnerability Name & Description: A clear explanation of what the vulnerability is and how it functions.

- Impact Assessment: An evaluation of what an attacker could achieve if they successfully exploited the flaw (e.g., Remote Code Execution, Data Leakage, Denial of Service).

- Severity Rating: A calculated risk score based on accessibility and potential damage.

Actionable Remediation Guidance The AI provides step-by-step mitigation instructions tailored to the specific environment.

Instead of generic advice, it outputs exact remediation steps, which may include:

- The precise commands required to update an outdated service.

- Specific configuration changes needed to close an open port safely.

- Code snippets or firewall rules to block unauthorized directory traversal.

4. AI Analysis Stage The AI analysis module processes the collected reconnaissance data and identifies possible

vulnerabilities and security risks.

The AI module examines: ☐ Open ports and exposed services ☐ Technologies with known vulnerabilities ☐ Sensitive directories ☐ Risky subdomains The module classifies vulnerabilities based on severity levels and generates security reports.

Publicly accessible sensitive directories are considered high-risk vulnerabilities.

1. Attack Surface Analysis The system generates an attack surface map representing the relationship between the target domain and discovered subdomains.

The AI module analyzes the attack surface to determine possible entry points for attackers.

2. Risk Correlation Analysis The AI module combines multiple reconnaissance results and correlates vulnerabilities to generate a more accurate security assessment.

26 For example: ☐ Open admin panel + outdated PHP version = Higher security risk ☐ Public database port + exposed backup directory = Critical vulnerability

4.5.2 Security Risk Classification

The AI analysis module classifies detected vulnerabilities into different severity levels based on the potential impact on the target system.

Risk Level Description Low Risk Minor security issues that do not directly compromise the system Medium

Risk Vulnerabilities that may be exploited under specific conditions High Risk Serious vulnerabilities that may lead to unauthorized access Critical Risk Severe vulnerabilities that may result in complete system compromise

Low Risk Vulnerabilities Low-risk vulnerabilities include: ☐ Information disclosure ☐ Missing security headers

☐ Minor configuration issues **Medium Risk Vulnerabilities** Medium-risk vulnerabilities include: ☐ Open SSH ports ☐ Exposed login pages ☐ Publicly accessible services **High Risk Vulnerabilities** High-risk vulnerabilities

include: ☐ Exposed admin panels 27 ☐ Outdated technologies ☐ Sensitive directories **Critical Risk**

Vulnerabilities Critical vulnerabilities include: ☐ Public database access ☐ Backup file exposure ☐

Misconfigured servers

4.5.3 AI-Based Decision Making

The AI module determines whether the target domain is secure or vulnerable based on the reconnaissance results and detected vulnerabilities.

The decision-making process includes: ☐ Number of discovered vulnerabilities ☐ Severity of vulnerabilities ☐ Open ports and services ☐ Technology risks ☐ Directory exposure level 28 **Testing Module**

2.1 Testing Approach

The Testing Module ensures that the Reconnaissance and Vulnerability Testing System works correctly and performs automated security scanning on target systems.

Testing verifies that all reconnaissance tools run successfully, data is collected properly, and the AI analysis module generates accurate security insights.

Testing also ensures that the system can handle user input, execute scanning tools, analyze the collected data, and present the results in a clear and structured format.

Objectives of Testing Verify that the reconnaissance tools are correctly installed and configured.

Ensure that the system accepts user input such as domain names or target IP addresses.

Confirm that scanning tools successfully collect information from the target system.

Validate that the AI analysis module processes scan results correctly.

Ensure that scanning results are properly stored and displayed in the visualization dashboard.

Types of Testing 1.

Installation Testing Verify that all required tools such as Subfinder, Nmap, WhatWeb, and Dirsearch are correctly installed.

Ensure that the Python environment and required libraries are configured properly.

Dependency and Path Verification The testing module runs environment checks before booting the Streamlit dashboard.

It verifies that the system's PATH variable can locate the executable binaries for every integrated tool.

☐ Subfinder (Go binary): Verifies the binary exists and checks for valid API keys (e.g., Chaos, Shodan) in

config.yaml file to ensure deep subdomain harvesting works.

29 ☐ Nmap (C-based binary): Runs a localized nmap --version check.

It also tests if the executing user has root/sudo privileges, which are required for advanced scanning techniques like SYN Stealth Scans (-sS).

☐ WhatWeb (Ruby package): Ensures the Ruby interpreter is present and capable of loading plugging databases.

☐ Dirsearch (Python package): Validates that the default wordlists (e.g., common.txt) are present in the designated paths.

2.Functional Testing Functional evaluates the system against its core business logic requirements,treating the application as a predictable“input-to-input”machine.

Ensure that the system successfully collects information such as open ports, technologies used, directories and subdomains.

3.Integration Testing Ensure that all modules work together smoothly: User Input Module Reconnaissance Tools Execution AI Analysis Module Result Logging Visualization Dashboard 4.Performance Testing Test the system performance when scanning multiple targets.

Monitor CPU and memory usage during reconnaissance and vulnerability scanning.

5.Security Testing Verify that the system safely performs reconnaissance without affecting the target system.

Ensure that scan results correctly identify potential vulnerabilities such as open ports, exposed directories, or outdated technologies.

30 2.1Test cases: 4.

Tools Used Subfinder → Used for discovering subdomains of the target domain.

Nmap → Used for port scanning and service detection.

WhatWeb → Used to identify web technologies such as CMS, frameworks, and servers.

Dirsearch → Used to discover hidden directories and files on the target website.

Python → Used to integrate reconnaissance tools and automate the scanning process.

AI Analysis Module → Used to analyze scan results and generate security insights.

5. Expected Outcomes The system should successfully accept target input (domain or IP address) from the user.

Reconnaissance tools should collect information such as subdomains, open ports, technologies, and directories.

The AI module should analyze the collected data and identify potential vulnerabilities.

Scan results should be stored and logged properly in the system.

31 The visualization dashboard should display the reconnaissance results clearly for the user 32 Fig 3 test case 33 1.

Performance of the Project Developed (So Far) Performance of the Developed Reconnaissance and Vulnerability Testing System The performance of the Reconnaissance and Vulnerability Testing System using AI was evaluated based on scanning efficiency, system resource utilization, and response time while performing different reconnaissance tasks such as subdomain discovery, port scanning, and technology detection.

1. Scanning Performance The system successfully performed reconnaissance tasks such as: Subdomain discovery using Subfinder Port scanning using Nmap Technology detection using WhatWeb Directory discovery using Dirsearch Each tool produced accurate results for the given target domain or IP address.

The AI analysis module processed the collected data and generated useful insights about possible vulnerabilities and security risks.

The system successfully combined outputs from multiple tools and displayed them in a structured format.

2. Resource Utilization CPU Usage During normal scans the CPU usage remained around 20–30%.

During intensive scanning such as Nmap port scans or directory discovery, CPU usage increased to approximately 50–60%.

Memory Usage The application used approximately 400–700 MB of RAM during normal operation.

34 Memory usage slightly increased when running multiple reconnaissance tools simultaneously.

Disk Usage Scan results and logs required minimal storage space.

Most results were stored in JSON or text format, which kept disk usage relatively low.

3. Response Time The system responded quickly to user input when a target domain or IP address was entered.

Scanning results were generated within a few seconds to a few minutes, depending on the type of scan.

The Streamlit dashboard displayed the results almost instantly after the scanning process completed.

4. Scalability The system works efficiently for small to medium-sized websites or domains.

When scanning larger targets with many subdomains or directories, the scanning time increases.

Performance can be improved by allocating more CPU and memory resources.

5. Reliability The application ran reliably in the Windows environment using Visual Studio Code without crashes.

Reconnaissance tools executed successfully when triggered from the Python script.

Results were consistently generated and displayed in the user interface.

2. AI-Driven Vulnerability Analysis Performance The AI-driven vulnerability analysis module was successfully tested using different target domains and reconnaissance results generated from Subfinder, Nmap, WhatWeb, and Dirsearch.

The AI module analyzed collected data and identified possible vulnerabilities based on detected technologies, open ports, hidden directories, and exposed services.

The AI analysis system successfully performed the following tasks: 35 □ Detection of risky open ports □ Identification of exposed directories □ Detection of outdated technologies □ Attack surface analysis using discovered subdomains □ Classification of vulnerabilities into severity levels The module generated intelligent security insights and helped determine whether the target domain was relatively secure or potentially vulnerable. The AI-assisted analysis reduced manual effort required for reviewing reconnaissance data and improved the speed of vulnerability assessment.

The system also successfully correlated multiple reconnaissance results to generate better security analysis.

For example, if an outdated technology and exposed admin directory were detected together, the system classified the target as a higher security risk.

3. Security Assessment Accuracy The proposed system successfully analyzed reconnaissance data and generated meaningful security assessments for tested domains.

The AI module correctly identified common security risks such as: □ Publicly accessible services □ Open ports □ Sensitive directories □ Outdated technologies □ Exposed administrative panels The generated results were compared with the actual outputs of reconnaissance tools, and the system produced consistent vulnerability insights in most cases.

The framework was able to classify vulnerabilities into different severity levels including low, medium, high, and critical risk categories.

The attack surface mapping feature also improved the visibility of relationships between the target domain and discovered subdomains.

This helped in understanding possible attack entry points and overall exposure of the target system.

4. Report Generation Performance The system successfully generated structured reconnaissance and vulnerability analysis reports in PDF format.

The generated reports included: □ Subdomain discovery results □ Open port details 36 □ Technology detection results □ Directory discovery results □ AI-generated vulnerability analysis □ Attack surface visualization The report generation process was completed within a short duration after scanning.

The generated reports were easy to understand and provided centralized documentation of reconnaissance and security testing results.

The Streamlit dashboard also displayed results graphically, allowing users to quickly analyze the security status of the target domain.

The integration of visualization and reporting improved the usability and effectiveness of the system.

5. Limitations The system depends on external reconnaissance tools, so scanning speed depends on those tools. Large-scale scans may take longer to complete.

AI analysis currently provides basic vulnerability insights and may require more advanced models for deeper analysis.

References

Nmap Official Documentation – Guide for network discovery and port scanning. Retrieved from:

<https://nmap.org/docs.html>

Subfinder Tool Documentation – ProjectDiscovery – Documentation for subdomain discovery tool used in reconnaissance.

Retrieved from:

<https://github.com/projectdiscovery/subfinder>

WhatWeb Official Documentation – Tool used for identifying technologies used on websites. Retrieved from:

<https://github.com/urbanadventurer/WhatWeb>

Dirsearch GitHub Repository – Directory and file brute-forcing tool used for discovering hidden web directories.

Retrieved from:

<https://github.com/maurosoria/dirsearch>

Python Official Documentation – Programming language used to develop and automate the reconnaissance system.

Retrieved from:

<https://docs.python.org/3/>

Streamlit Documentation – Used for building the graphical interface and visualization dashboard for the project.

Retrieved from:

<https://docs.streamlit.io>

Visual Studio Code Documentation – Development environment used for coding and managing the project.

Retrieved from:

<https://code.visualstudio.com/docs>

Microsoft PowerShell Documentation – Used for running commands and executing tools in the Windows environment.

Retrieved from:

<https://learn.microsoft.com/en-us/powershell/>

OpenAI ChatGPT – Used for assistance in understanding concepts, debugging code, and improving project documentation.

Retrieved from:

<https://chat.openai.com>

Perplexity AI – Used as a research tool for gathering cybersecurity and reconnaissance-related information.

Retrieved from:

<https://www.perplexity.ai>

42

43

.

44

45

46

47

48

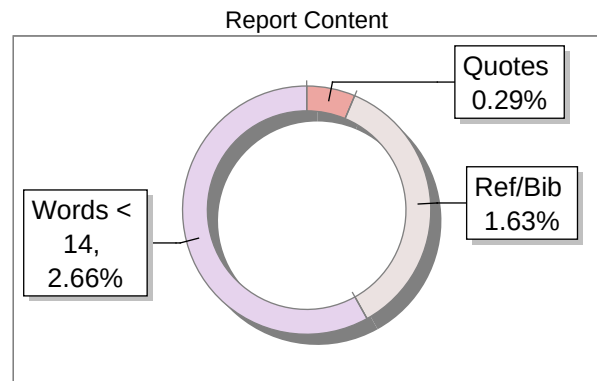
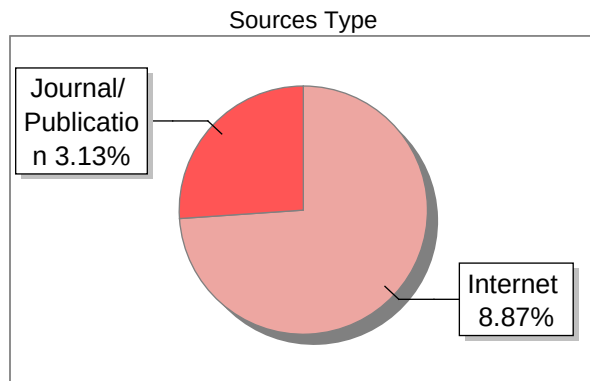
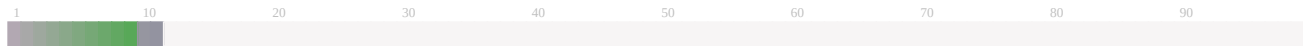
49

Submission Information

Author Name	G11
Title	Cyber
Paper/Submission ID	5819980
Submitted by	amanpreet_brar@gndec.ac.in
Submission Date	2026-06-03
Total Pages, Total Words	62, 10960
Document type	Project Work

Result Information

Similarity **12 %**



Exclude Information

Quotes	Excluded
References/Bibliography	Excluded
Source: Excluded < 14 Words	Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

